Instituto Tecnológico de Costa Rica
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación
IC-6600 Principios de Sistemas Operativos

Tarea #3
Mr. Meeseeks

Asignación:	Tarea #3 — Mr. Meeseeks
Estudiante:	José Julián Monge Brenes
Número de carné:	2024247024
Fecha de entrega:	15 de octubre del 2026
Periodo:	II-Semestre de 2026
Profesor:	Ing. Esteban Arias Méndez, M.Sc.
Curso:	IC-6600 Principios de Sistemas Operativos

Abstract

This report presents the design and implementation of Mr. Meeseeks Box, a C application for Linux that uses operating-system processes to handle three categories of user requests. Text requests are addressed through a probabilistic simulation in which tasks become easier across successive generations of Meeseeks. Arithmetic and logical expressions are evaluated with TinyExpr, whereas external programs are launched and checked through their exit status. The application relies on `fork()` for process creation, pipes for private communication and propagation of results, and semaphores with shared memory to protect process counters. It also incorporates limits on process creation and execution time, orderly cancellation, and a controlled experimental mode. The report presents the implementation, execution evidence, measured results, and comparative experiments involving task difficulty, resource limits, execution times, and cancellation mechanisms.

Índice

1. Introducción **7**

1.1. Objetivo general . . . 7

1.2. Objetivos específicos . . . 7

2. Descripción del problema y alcance **7**

3. Diseño e implementación de la solución (0x01) **8**

3.1. Organización general del código . . . 8

3.2. Representación de solicitudes y resultados . . . 11

3.3. Creación de procesos y datos de identidad . . . 12

3.4. Comunicación interprocesos mediante pipes . . . 13

3.5. Caso A: solicitudes textuales simuladas . . . 13

3.5.1. Captura y duración . . . 13

3.5.2. Modelo probabilístico y simplificación por generaciones . . . 14

3.5.3. Árbol de procesos y resolución en cascada . . . 15

3.6. Caso B: operaciones aritméticas y lógicas . . . 17

3.7. Caso C: ejecución de programas externos . . . 18

3.8. Sincronización y recursos compartidos . . . 20

3.9. Contingencia y caos planetario . . . 21

3.10. Medición de tiempos y generación del informe . . . 21

4. Resultados de ejecución: mínimo tres corridas por caso **24**

4.1. Caso A: solicitudes textuales . . . 24

4.1.1. Corrida A1: tarea fácil . . . 24

4.1.2. Corrida A2: tarea intermedia . . . 25

4.1.3. Corrida A3: tarea difícil . . . 28

4.1.4. Corrida extra A4: solicitud imposible . . . 31

4.2. Caso B: operaciones aritméticas y lógicas . . . 33

4.2.1. Corrida B1: expresión aritmética . . . 33

4.2.2. Corrida B2: expresión lógica . . . 34

4.2.3. Corrida B3: expresión inválida . . . 36

4.3. Caso C: ejecución de programas externos . . . 37

4.3.1. Corrida C1: programa exitoso . . . 37

4.3.2. Corrida C2: programa que retorna error . . . 38

4.3.3. Corrida C3: programa interrumpido por plazo . . . 39

5. Experimentos comparativos y análisis (0x02) **42**

5.1. Experimento E1: influencia de la facilidad . . . 42

5.2. Experimento E2: comparación de operaciones y programas externos . . . 45

5.3. Experimento E3: contingencia por procesos y caos planetario . . . 45

5.4. Experimento E4: ejecución después del plazo normal 46

6. Uso de Git y organización de la entrega **48**

7. Conclusiones **50**

8. Recomendaciones y trabajo futuro **50**

A. Código fuente **51**

Referencias Bibliográficas **52**

Índice de figuras

1. Estructura general de funciones del programa, elaborada a partir de `main.c`. 8
2. Función `box()`, inicio. 9
3. Función `box()`: menú. 9
4. Función `box()`: recepción solicitudes y `main()`. 10
5. Estructuras utilizadas para las solicitudes, resultados y procesos hijos. 11
6. Creación y recolección de Meeseeks mediante `fork()` y `waitpid()`. 12
7. Sentido de la comunicación privada entre un padre y un hijo. 13
8. Lectura de facilidad y duración de la solicitud textual. 13
9. Funciones del algoritmo de facilidad, probabilidad y creación de hijos. 15
10. Propagación simplificada del éxito hacia el padre y cancelación de un hermano. 15
11. Cancelación de ramas hermanas en `avisar_hijos()`. 16
12. Recepción del resultado ganador en `esperar_hijos()`. 16
13. Evaluación de expresiones mediante TinyExpr. 17
14. Función `procesar_programa()`: preparación y creación del proceso externo. 18
15. Función `procesar_programa()`: espera, tiempo máximo y resultado final. 19
16. Función `ejecutar_programa_hijo()`. 19
17. Inicialización del control compartido y acceso protegido al semáforo. 20
18. Reserva de procesos en `reservar_meeseeks()`. 20
19. Configuración del tiempo máximo y control del presupuesto. 21
20. Función `medir_cpu_total()`. 22
21. Función `procesar_solicitud()`: preparación y medición inicial. 22
22. Función `procesar_solicitud()`: consolidación del resultado y estadísticas. 23
23. Caso A, corrida 1: creación y atención de una tarea fácil. 24
24. Caso A, corrida 1: resultado e informe final. 25
25. Caso A, corrida 2: creación de hijos para una tarea intermedia. 26
26. Caso A, corrida 2: ganador, notificación y cancelación de hermanos. 26
27. Caso A, corrida 2: informe de la atención intermedia. 27
28. Caso A, corrida 3: primer nivel y creación de cinco hijos. 28
29. Caso A, corrida 3: generaciones y comunicación en cascada. 29
30. Caso A, corrida 3: estadísticas de la tarea difícil. 30
31. Contingencia para una solicitud imposible. 31
32. Informe de la solicitud imposible. 32
33. Caso B, corrida 1: expresión aritmética y Meeseeks. 33
34. Caso B, corrida 1: resultado e informe. 33
35. Caso B, corrida 2: expresión lógica. 34
36. Caso B, corrida 2 y 3: resultado e informe de error. 35
37. Caso B, corrida 3: expresión inválida. 36
38. Prueba complementaria B4: división por cero. 36

39. Caso C, corrida 1: ejecución de `pwd`. 37
40. Caso C, corrida 1: estado del programa e informe. 37
41. Caso C, corrida 2: programa con código de error. 38
42. Caso C, corrida 2: informe de programa fallido. 39
43. Caso C, corrida 3: ejecución y cancelación por plazo. 40
44. Caso C, corrida 3: informe del caos planetario. 40
45. Prueba complementaria C4: programa inexistente. 41
46. Informe de las tres solicitudes con facilidad 95. 43
47. Informe de las tres solicitudes con facilidad 65. 43
48. Informe de las tres solicitudes con facilidad 25. 44
49. Informe de las tres solicitudes imposibles con facilidad 0. 44
50. Informe final de la corrida E4.1. 47
51. Informe final de la corrida E4.2. 47
52. Informe final de la corrida E4.3. 47
53. Informe final de la corrida E4.4. 48
54. Historial de commits del repositorio en GitHub. 49

Índice de cuadros

1. Tipos de solicitudes atendidas por la Box. 7
2. Reglas para solicitar ayuda según la facilidad original. 14
3. Comparación de solicitudes textuales según la facilidad. 42
4. Comparación entre operaciones y programas externos. 45
5. Comparación de los mecanismos de contingencia. 46
6. Resultados de las ejecuciones con y sin modo experimental. 46

1. Introducción

La creación y administración de procesos es una responsabilidad central de los sistemas operativos. En Linux, una aplicación puede crear procesos hijos, intercambiar información entre ellos y coordinar su terminación mediante interfaces como `fork()`, `pipe()` y `waitpid()` [1, 2]. También es necesario controlar el acceso a los recursos compartidos para evitar condiciones de carrera.

En esta tarea se desarrolló una *Box de Mr. Meeseeks*. La Box recibe solicitudes del usuario y crea un proceso Meeseeks para atender cada una. Algunas solicitudes se simulan utilizando un modelo probabilístico, otras se resuelven al interpretar expresiones aritméticas o lógicas, o mediante la ejecución de programas externos de Linux. Cuando una tarea textual requiere ayuda, el Meeseeks puede crear hijos y formar un árbol de procesos que trabaja de manera concurrente.

La implementación se desarrolló en lenguaje C sobre GNU/Linux y utiliza pipes para transmitir solicitudes y resultados, semáforos para proteger contadores compartidos y mecanismos de control para evitar el agotamiento de recursos. El reporte presenta las decisiones de diseño, las pruebas realizadas y el análisis comparativo de sus resultados.

1.1. Objetivo general

Implementar y evaluar una aplicación en C que simule la atención de solicitudes mediante procesos Meeseeks, empleando mecanismos reales de creación de procesos, comunicación interprocesos, sincronización y control de recursos en Linux.

1.2. Objetivos específicos

- Crear y gestionar procesos hijos mediante `fork()` y `waitpid()`.
- Transmitir solicitudes, respuestas y avisos de cancelación mediante pipes.
- Coordinar contadores compartidos mediante semáforos administrados por el sistema operativo.
- Implementar la simulación probabilística, el procesamiento de expresiones y la ejecución de programas externos.
- Establecer límites para controlar tareas difíciles o imposibles.
- Comparar los resultados de los distintos escenarios mediante mediciones y pruebas reproducibles.

2. Descripción del problema y alcance

La aplicación presenta un menú con tres tipos de solicitudes. La selección es explícita, evitando que el programa deba adivinar si el texto del usuario es una tarea simulada, un cálculo o un comando externo. Se permiten solicitudes consecutivas y, al finalizar, se produce un informe con los resultados.

Cuadro 1: Tipos de solicitudes atendidas por la Box.

Tipo	Entrada de ejemplo	Forma de atención
A. Textual	`Hola, quiero un unicornio`	Simulación probabilística, puede crear generaciones de Meeseeks.
B. Cálculo	`2 + 3 * 4`	Interpretación de operaciones aritméticas y lógicas por un Meeseeks.
C. Programa	`pwd`	Ejecución externa y revisión del código de terminación.

3. Diseño e implementación de la solución (0x01)

3.1. Organización general del código

Se utilizó un solo archivo fuente propio, `src/main.c`, organizado en funciones con responsabilidades específicas. La función `main()` muestra el título de la aplicación y delega el control a `box()`. Esta última administra el menú, captura la solicitud y guarda el resultado que posteriormente se incluye en el informe. La biblioteca externa TinyExpr permanece en la carpeta `vendor/tinyexpr`; se emplea para interpretar las expresiones del caso B.

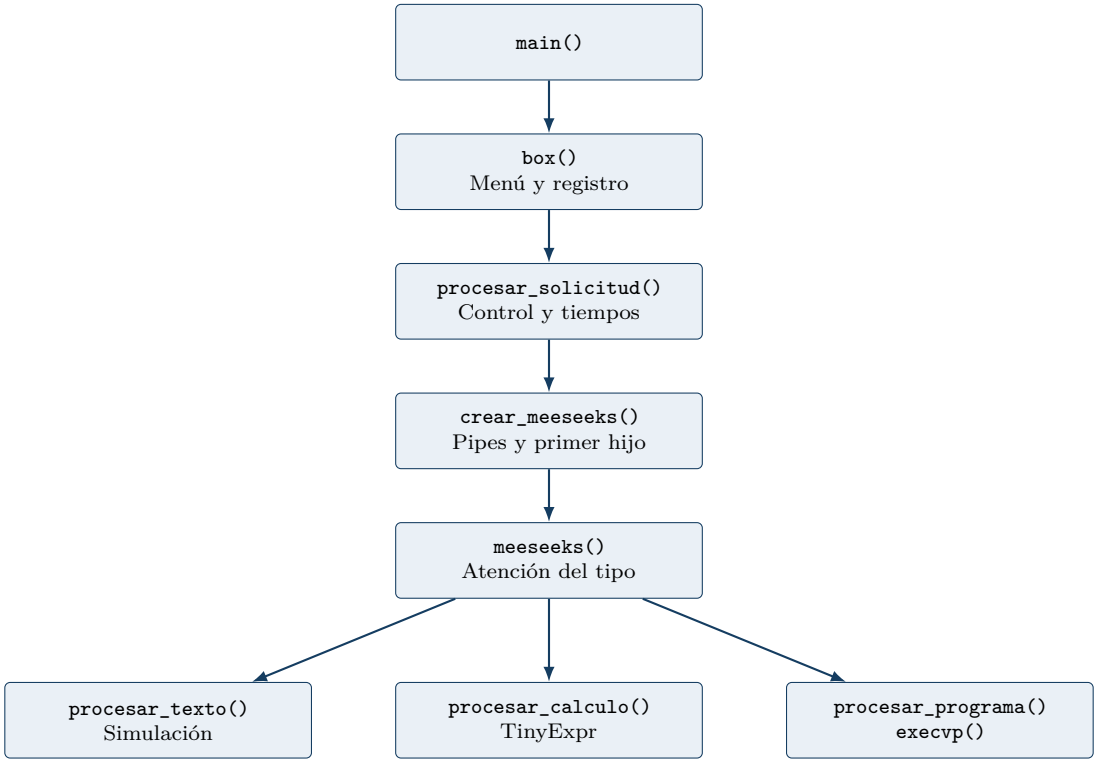


Figura 1: Estructura general de funciones del programa, elaborada a partir de `main.c`.

```c
/* Controla el menú, captura y procesa solicitudes, muestra el informe y libera el historial. */
static int box(void)
{
    char *texto = NULL;
    size_t capacidad = 0;
    RegistroSolicitud *historial = NULL;
    size_t total_solicitudes = 0;
    int resultado = EXIT_SUCCESS;

    Configuracion configuracion;
    if (!configurar_box(&configuracion))
        return EXIT_FAILURE;

    printf("Box: limites por solicitud: %u Meeseeks, %.3f segundos.\n", configuracion.max_procesos, configuracion.plazo_segundos);

    /* Un pipe sin lector debe producir EPIPE, sin terminar abruptamente la Box. */
    struct sigaction accion = {0};
    accion.sa_handler = SIG_IGN;

    sigemptyset(&accion.sa_mask);
    if (sigaction(SIGPIPE, &accion, NULL) == -1)
    {
        perror("Error al configurar SIGPIPE");
        return EXIT_FAILURE;
    }

    srand(configuracion.semilla);
    printf("Box: modo %s; limite efectivo %.3f segundos.\n",
           configuracion.modo_experimental ? "experimental" : "normal",
           plazo_efectivo(&configuracion));
```

Figura 2: Función `box()`, inicio.

```c
static int box(void)
    while (1)
    {
        puts("\n--- MENU PRINCIPAL ---");
        puts("1. Solicitud textual (simulacion)");
        puts("2. Calculo aritmetico o logico");
        puts("3. Ejecucion de programa externo");
        puts("0. Salir");

        int opcion = leer_opcion();

        if (opcion == -2)
            break;

        if (opcion < 0 || opcion > 3)
        {
            puts("Opcion invalida. Seleccione un numero entre 0 y 3.");
            continue;
        }
        else if (opcion == 0)
            break;

        if (!leer_solicitud(&texto, &capacidad))
        {
            if (feof(stdin) || ferror(stdin))
                break;

            puts("Solicitud invalida: no puede estar vacia ni superar 255 caracteres.");
            continue;
        }

        Solicitud solicitud = {0};
        solicitud.tipo = (TipoSolicitud)opcion;
        strcpy(solicitud.contenido, texto);

        if (solicitud.tipo == TIPO_TEXTO &&
            !capturar_datos_textuales(&solicitud))
        {
            if (feof(stdin) || ferror(stdin))
                break;

            puts("Box: solicitud cancelada.");
            continue;
        }

        solicitud.semilla = derivar_semilla(configuracion.semilla, (unsigned int)(total_solicitudes + 1));
        if (!registrar_solicitud(&historial, &total_solicitudes, &solicitud))
        {
            perror("Error al reservar el registro de la solicitud");
            resultado = EXIT_FAILURE;
```

Figura 3: Función `box()`: menú.

```c
static int box(void)
    while (1)
        if (!registrar_solicitud(&historial, &total_solicitudes, &solicitud))

        mostrar_solicitud(&solicitud, total_solicitudes);

        RegistroSolicitud *registro = &historial[total_solicitudes - 1];
        bool procesada = procesar_solicitud(registro, &configuracion);
        mostrar_resultado(&registro->resultado);
        mostrar_tiempos(registro);

        if (!procesada || registro->resultado.caos_planetario)
        {
            resultado = EXIT_FAILURE;
            break;
        }
    }

    if (ferror(stdin))
    {
        perror("Error al leer la entrada");
        resultado = EXIT_FAILURE;
    }
    mostrar_informe(historial, total_solicitudes);
    puts("Box: ¡Hasta luego!");
    free(texto);
    free(historial);
    return resultado;

}

/* Muestra el título y delega el control a box. */
int main(void)
{
    printf("=== Box de Mr. Meeseeks === \n");

    return box();
}
```

Figura 4: Función `box()`: recepción solicitudes y `main()`.

3.2. Representación de solicitudes y resultados

El programa utiliza `struct` y `enum` para organizar los datos [3]. `TipoSolicitud` distingue los tres casos, y `Solicitud` agrupa el contenido textual, la facilidad y las opciones de duración. La estructura `Resultado` contiene el estado de terminación, la identidad del Meeseeks, la identidad del ganador y detalles para el informe. Por su parte, `HijoMeeseeks` conserva el PID y los descriptores de los pipes asociados a cada hijo directo.

La estructura `ControlCompartido` contiene el semáforo y los contadores que deben ser visibles para distintos procesos. Las solicitudes propiamente dichas no se transmiten a través de esa memoria, sino que se envían por pipes.

```c
typedef struct
{
    TipoSolicitud tipo;
    char contenido[MAX_TEXTO];
    double facilidad;
    bool facilidad_aleatoria;
    bool tiempo_aleatorio;
    double tiempo_segundos;
    unsigned int semilla;
} Solicitud;

/* Respuesta privada del Meeseeks; no contiene punteros a memoria del hijo. */
typedef struct
{
    EstadoResultado estado;
    pid_t pid;
    pid_t ppid;
    unsigned int nivel;
    unsigned int instancia;
    unsigned int meeseeks_creados;
    unsigned int profundidad_maxima;
    bool caos_planetario;
    IdentidadMeeseeks ganador;
    double tiempo_programa_segundos;
    char detalle[MAX_TEXTO];
} Resultado;

typedef struct
{
    pid_t pid;
    int envio;
    int recepcion;
} HijoMeeseeks;
```

Figura 5: Estructuras utilizadas para las solicitudes, resultados y procesos hijos.

3.3. Creación de procesos y datos de identidad

Para cada solicitud se crea inicialmente un proceso mediante `fork()`. El proceso hijo recibe el contenido mediante su pipe privado y ejecuta la función `meeseeks()`. Su identidad se obtiene con `getpid()` y `getppid()`. Cuando se crean hijos adicionales, se incrementa el nivel y se asigna una instancia con respecto a su mismo padre.

La función `waitpid()` permite que cada padre espere y recoja a sus hijos, evitando dejar procesos zombis [2].

```
static bool iniciar_hijo(const Solicitud *solicitud, unsigned int nivel, unsigned int instancia, int en
    if (pipe(respuesta) == -1)
        close(peticion[1]);
        errno = error;
        return false;
    }

    pid_t pid = fork();
    if (pid == -1)
    {
        int error = errno;
        close(peticion[0]);
        close(peticion[1]);
        close(respuesta[0]);
        close(respuesta[1]);
        errno = error;
        return false;
    }
    if (pid == 0)
    {
        close(peticion[1]);
        close(respuesta[0]);
        /* El nieto no debe conservar canales de sus tios ni de sus abuelos. */
        cerrar_descriptor(&entrada_padre);
        cerrar_descriptor(&salida_padre);
        for (unsigned int i = 0; i < cantidad_hermanos; i++)
        {
            cerrar_descriptor(&hermanos[i].envio);
            cerrar_descriptor(&hermanos[i].recepcion);
        }
        int estado = meeseeks(peticion[0], respuesta[1], nivel, instancia, control);
        if (fflush(stdout) == EOF)
            estado = EXIT_FAILURE;
        _exit(estado);
    }

    hijo->pid = pid;
    hijo->envio = peticion[1];
    hijo->recepcion = respuesta[0];
    close(peticion[0]);
    close(respuesta[1]);
    if (!escribir_completo(hijo->envio, solicitud, sizeof(*solicitud)))
    {
        int error = errno;
        cerrar_descriptor(&hijo->envio);
        errno = error;
        return false;
    }
    return true;
}
```

Figura 6: Creación y recolección de Meeseeks mediante `fork()` y `waitpid()`.

3.4. Comunicación interprocesos mediante pipes

Cada relación padre–hijo utiliza dos pipes. Por un canal, el padre envía la solicitud inicial y, posteriormente, posibles avisos de resolución o cancelación. Por el otro lado, el hijo transmite a su padre la estructura con el resultado de su trabajo. Los descriptores que no corresponden a cada proceso se cierran para no mantener extremos de pipes abiertos innecesariamente [1].

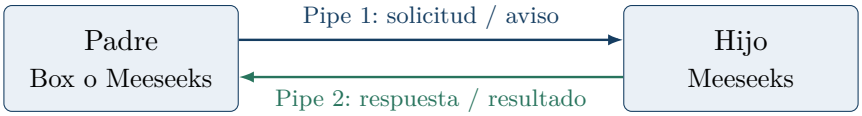


Figura 7: Sentido de la comunicación privada entre un padre y un hijo.

Las funciones `leer_completo()` y `escribir_completo()` manejan transferencias parciales para asegurar que se transmitan los mensajes completos.

3.5. Caso A: solicitudes textuales simuladas

3.5.1. Captura y duración

En el caso A, el usuario define una facilidad entre 0 y 100 de forma manual o permite que el programa la genere aleatoriamente. Se dice facilidad porque un valor elevado significa que la tarea es más sencilla. La duración simulada puede ser manual o aleatoria en el intervalo de 0.5 a 5 segundos.

La espera se realiza de modo que el proceso pueda recibir avisos de cancelación por pipe.

```c
/* Configura facilidad y duración; false indica cancelación, fin de entrada o error. */
static bool capturar_datos_textuales(Solicitud *solicitud)
{
    puts("\n--- CONFIGURACION DE TAREA TEXTUAL ---");
    puts("Facilidad: 0 = imposible, 100 = muy facil.");
    puts("1. Ingresar facilidad manualmente");
    puts("2. Generar facilidad aleatoriamente");
    puts("0. Cancelar solicitud");

    int opcion = leer_modalidad();

    if (opcion <= 0)
        return false;

    solicitud->facilidad_aleatoria = opcion == 2;

    if (solicitud->facilidad_aleatoria)
        solicitud->facilidad = 100.0 * aleatorio_uniforme();

    else if (!leer_numero_en_rango("Facilidad (0 a 100): ", "La facilidad debe estar entre 0 y 100.", 100.0, true, &solicitud->facilidad))
        return false;

    puts("\nDuracion de la simulacion:");
    puts("1. Definida por el usuario");
    puts("2. Aleatoria (entre 0.5 y 5 segundos)");
    puts("0. Cancelar solicitud");

    opcion = leer_modalidad();
    if (opcion <= 0)
        return false;

    solicitud->tiempo_aleatorio = opcion == 2;
    solicitud->tiempo_segundos = 0.0;

    return solicitud->tiempo_aleatorio ||  leer_numero_en_rango("Duracion en segundos (> 0): ", "La duracion debe ser mayor que cero.", INFINITY,
    false, &solicitud->tiempo_segundos);
}
```

Figura 8: Lectura de facilidad y duración de la solicitud textual.

3.5.2. Modelo probabilístico y simplificación por generaciones

Para las solicitudes textuales se implementó un modelo probabilístico basado en la facilidad de cada tarea, utilizando valores entre 0 y 100. Cuanto mayor sea la facilidad, mayores serán las posibilidades de que un Meeseeks resuelva la solicitud.

Cuando un Meeseeks no consigue resolver una tarea, puede solicitar ayuda creando nuevos procesos. Para representar que las siguientes generaciones tienen más posibilidades de éxito, se decidió reducir un 20 % de la dificultad restante en cada generación.

Por ejemplo, una solicitud con facilidad inicial de 25 tiene una dificultad restante de 75. Al reducir esa dificultad un 20 %, la facilidad aumenta a 40 en la segunda generación y a 52 en la tercera.

Para determinar si la tarea se resuelve, el programa calcula una probabilidad a partir de la facilidad alcanzada. En este modelo, la facilidad expresada como proporción se multiplica por sí misma. Así, una facilidad de 25 representa una probabilidad de éxito del 6.25 %, mientras que una facilidad de 40 representa un 16 % y una de 52 representa un 27.04 %.

Cuando la facilidad efectiva supera 85, la probabilidad de éxito es del 100 %. Por otra parte, las solicitudes con facilidad 0 se consideran imposibles, por lo que nunca aumentan su facilidad ni pueden resolverse.

De esta manera, las tareas difíciles suelen necesitar más procesos, mientras que las más sencillas pueden resolverse con menos recursos.

Cuadro 2: Reglas para solicitar ayuda según la facilidad original.

Facilidad original	Condición	Hijos creados cuando se requiere ayuda
$0 \leq F \leq 45$	El intento no resuelve la tarea.	Al menos cinco hijos; en el programa se crea un grupo de cinco.
$45 < F \leq 85$	El intento no resuelve la tarea.	Al menos dos hijos; en el programa se crea un grupo de dos.
$85 < F \leq 100$	La tarea se considera sencilla.	La resuelve el Meeseeks actual, sin crear hijos.

La cantidad de hijos se determina con la facilidad *original*, mientras que la probabilidad de éxito se calcula con la facilidad *efectiva* de cada generación.

```c
/*
    El aging reduce un 20% de la dificultad restante por generación
    La facilidad cero nunca cambia.
 */
static double facilidad_por_generacion(double facilidad, unsigned int nivel)
{
    if (facilidad == 0.0 || nivel == 1)
        return facilidad;
    return 100.0 - (100.0 - facilidad) * pow(0.8, (double)(nivel - 1));
}

/* Usa (facilidad efectiva / 100)^2, en el caso de 85 o más la probabilidad es 1. */
static double probabilidad_resolver(double facilidad, unsigned int nivel)
{
    double efectiva = facilidad_por_generacion(facilidad, nivel);
    return efectiva > 85.0 ? 1.0 : pow(efectiva / 100.0, 2.0);
}

/* Según la facilidad original se necesitan cinco hijos hasta 45, dos hasta 85 y cero sobre 85. */
static unsigned int ayuda_necesaria(double facilidad)
{
    return facilidad > 85.0 ? 0 : (facilidad <= 45.0 ? 5 : 2);
}
```

Figura 9: Funciones del algoritmo de facilidad, probabilidad y creación de hijos.

3.5.3. Árbol de procesos y resolución en cascada

Cuando un hijo encuentra la solución, comunica el resultado a su padre. El padre informa a los demás hijos que la misión ha sido resuelta y que deben interrumpir sus subárboles. Los procesos se despiden y cada padre recoge a sus descendientes antes de transmitir la respuesta hacia el nivel superior. Este procedimiento continúa hasta que el Meeseeks inicial informa a la Box la identidad del ganador.

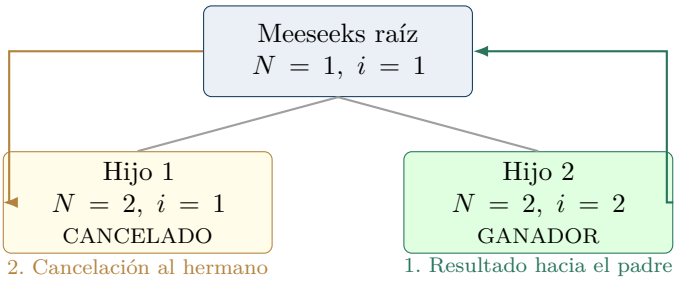


Figura 10: Propagación simplificada del éxito hacia el padre y cancelación de un hermano.

En árboles con más niveles, cada hijo aplica la misma lógica a sus propios descendientes. La función `poll()` permite esperar mensajes de varios canales sin bloquear permanentemente la atención a los demás hijos.

```c
/* Envía por pipe el ganador o la cancelación a cada hijo; EPIPE indica que ya cerró su lector. */
static bool avisar_hijos(HijoMeeseeks hijos[], unsigned int cantidad, const AvisoCancelacion *aviso, const Resultado *padre)
{
    bool correcto = true;
    for (unsigned int i = 0; i < cantidad; i++)
    {
        if (hijos[i].pid == 0 || hijos[i].envio < 0)
            continue;
        if (!escribir_completo(hijos[i].envio, aviso, sizeof(*aviso)))
        {
            /* Un hijo que ya envio su resultado puede haber cerrado el lector. */
            if (errno != EPIPE)
                correcto = false;
            cerrar_descriptor(&hijos[i].envio);
        }
        else if (aviso->motivo == AVISO_RESUELTO)
            mensaje_meeseeks(padre, "Aviso por pipe a hijo PID %ld: gano PID %ld; deten tu subarbol.", (long)hijos[i].pid,
                             (long)aviso->ganador.pid);
        else
            mensaje_meeseeks(padre, "Aviso por pipe a hijo PID %ld: cancelar por %s.", (long)hijos[i].pid, aviso->motivo == AVISO_PLAZO ? "plazo" :
                             (aviso->motivo == AVISO_INTERRUPCION ? "interrupcion" : "error"));
    }
    return correcto;
}
```

Figura 11: Cancelación de ramas hermanas en `avisar_hijos()`.

```c
static bool esperar_hijos(int entrada, HijoMeeseeks hijos[], unsigned int cantidad, ControlCompartido *control, Resultado *resultado,
    while (pendientes > 0)
        for (unsigned int i = 0; i < cantidad; i++)
            if (hijos[i].pid == 0 || canales[i + 1].revents == 0)
                continue;

            if (canales[i + 1].revents & (POLLERR | POLLNVAL))
            {
                correcto = false;
                aviso.motivo = AVISO_ERROR;
                (void)cancelar_hijos(hijos, cantidad, &aviso, resultado);
            }

            Resultado respuesta = {0};
            bool recogido = recoger_hijo(&hijos[i], control, &respuesta, resultado);
            pendientes--;

            if (!recogido)
            {
                correcto = false;
                aviso.motivo = AVISO_ERROR;
                (void)cancelar_hijos(hijos, cantidad, &aviso, resultado);
            }
            else if (aviso.motivo == AVISO_NINGUNO && respuesta.estado == RESULTADO_COMPLETADO)
            {
                resultado->estado = RESULTADO_COMPLETADO;
                resultado->ganador = respuesta.ganador;
                establecer_detalle(resultado, "%s", respuesta.detalle);
                aviso = (AvisoCancelacion){AVISO_RESUELTO, respuesta.ganador};
                mensaje_meeseeks(resultado, "Hijo informo por pipe: gano PID %ld. Aviso a sus hermanos.",
                                 (long)respuesta.ganador.pid);
                if (!avisar_hijos(hijos, cantidad, &aviso, resultado))
                    correcto = false;
            }
            else if (aviso.motivo == AVISO_NINGUNO && respuesta.caos_planetario)
            {
                aviso.motivo = AVISO_PLAZO;
                if (!cancelar_hijos(hijos, cantidad, &aviso, resultado))
                    correcto = false;
            }
        }
    }

    if (aviso.motivo == AVISO_NINGUNO)
    {
        resultado->estado = RESULTADO_NO_COMPLETADO;
        establecer_detalle(resultado, "Ningun descendiente resolvio la solicitud dentro del presupuesto.");
    }
    return correcto;
}
```

Figura 12: Recepción del resultado ganador en `esperar_hijos()`.

3.6. Caso B: operaciones aritméticas y lógicas

El programa utiliza la biblioteca *TinyExpr* para interpretar la expresión ingresada como texto [4]. El caso B requiere un solo Meeseeks, por lo que el resultado de la biblioteca se registra en la estructura `Resultado` y se devuelve por pipe. Se admiten operaciones numéricas y lógicas dentro de la sintaxis soportada por la versión incluida en el repositorio.

Según la documentación del proyecto, la implementación considera operadores aritméticos, paréntesis, comparaciones y operadores lógicos. Por trabajar con `double`, los resultados lógicos se representan con 0 o 1. Se detectan expresiones inválidas y situaciones numéricas como la división por cero. La evaluación lógica no garantiza cortocircuito, por ejemplo, una expresión que incluya una división por cero en su segundo operando puede reportar error aunque el resultado lógico pudiera decidirse por el primero.

```c
static void procesar_calculo(const char *expresion, Resultado *resultado)
    while (*actual != '\0')
        if (isalpha((unsigned char)*actual) || *actual == '_')
        {
            establecer_detalle(resultado, "Solo se admite admite numeros y operadores, sin funciones ni variables.");
            return;
        }

        actual++;
    }

    /* TinyExpr tambien evalua operaciones mientras compila la expresion. */
    if (feclearexcept(FE_ALL_EXCEPT) != 0)
    {
        establecer_detalle(resultado, "No se pudo preparar la deteccion de errores numericos.");
        return;
    }

    int error = 0;
    errno = 0;
    double valor = te_interp(expresion, &error);
    int error_numerico = errno;
    int excepciones = fetestexcept(FE_DIVBYZERO | FE_INVALID | FE_OVERFLOW);

    if (error < 0)
    {
        establecer_detalle(resultado, "Memoria insuficiente para interpretar la expresion.");
    }
    else if (error > 0)
    {
        establecer_detalle(resultado, "Expresion invalida cerca de la posicion %d (contada desde 1).", error);
    }
    else if (excepciones & FE_DIVBYZERO)
    {
        establecer_detalle(resultado, "Division por cero.");
    }
    else if ((excepciones & FE_INVALID) || error_numerico == EDOM)
    {
        establecer_detalle(resultado, "Operacion indefinida: por ejemplo 0/0, resto entre cero o potencia fuera de dominio.");
    }
    else if ((excepciones & FE_OVERFLOW) || error_numerico == ERANGE || !isfinite(valor))
    {
        establecer_detalle(resultado, "Resultado fuera del rango numerico admitido.");
    }
    else
    {
        resultado->estado = RESULTADO_COMPLETADO;
        establecer_detalle(resultado, "Resultado: %.15g", valor);
    }
}
```

Figura 13: Evaluación de expresiones mediante TinyExpr.

3.7. Caso C: ejecución de programas externos

En esta categoría, un Meeseeks prepara los argumentos del comando, crea un proceso para ejecutar el programa y utiliza `execvp()` para sustituir la imagen de ejecución. El Meeseeks espera su terminación mediante `waitpid()` y revisa el código de salida, un valor cero indica éxito, un valor distinto de cero indica que el programa no terminó exitosamente. También se distinguen fallos de lanzamiento y terminaciones provocadas por señales.

La ejecución no interpreta operadores de shell como pipes o redirecciones, se utilizan argumentos del ejecutable en lugar de enviar directamente el texto a una shell. Para evitar que los programas externos consuman entradas del menú, su entrada estándar se redirige a `/dev/null`. Si vence el plazo, se solicita su terminación y se espera a que el proceso finalice.

```c
/* Crea el proceso externo, mide su duración y distingue fallo de exec, código de salida o señal. */
static void procesar_programa(const char *contenido, int respuesta, int entrada, ControlCompartido *control, Resultado *resultado)
{
    resultado->estado = RESULTADO_NO_COMPLETADO;
    char comando[MAX_TEXTO];
    strcpy(comando, contenido);
    char *argumentos[MAX_ARGUMENTOS + 1];

    if (!separar_argumentos(comando, argumentos, MAX_ARGUMENTOS + 1, resultado))
        return;

    if (fflush(stdout) == EOF)
    {
        establecer_detalle(resultado, "Error al mostrar la salida antes de ejecutar: %s.", strerror(errno));
        return;
    }

    int errores[2];
    if (pipe(errores) == -1)
    {
        establecer_detalle(resultado, "Error al crear el pipe de ejecucion: %s.", strerror(errno));
        return;
    }

    /* Un exec correcto cierra el escritor. */
    struct timespec inicio;
    if (fcntl(errores[1], F_SETFD, FD_CLOEXEC) == -1 ||
        clock_gettime(CLOCK_MONOTONIC, &inicio) == -1)
    {
        establecer_detalle(resultado, "Error al preparar la ejecucion: %s.", strerror(errno));
        close(errores[0]);
        close(errores[1]);
        return;
    }

    pid_t pid = fork();
    if (pid == -1)
    {
        establecer_detalle(resultado, "Error al crear el proceso del programa: %s.", strerror(errno));
        close(errores[0]);
        close(errores[1]);
        return;
    }

    if (pid == 0)
    {
        close(errores[0]);
        ejecutar_programa_hijo(argumentos, errores[1], respuesta, entrada);
    }
```

Figura 14: Función `procesar_programa()`: preparación y creación del proceso externo.

```c
static void procesar_programa(const char *contenido, int respuesta, int entrada, ControlCompartido *control, Resultado *resultado)
    if (setpgid(pid, pid) == -1 && errno != EACCES && errno != ESRCH)
        close(errores[0]);
        establecer_detalle(resultado, "Error al crear el grupo del programa: %s.", strerror(error));
        return;
    }

    int estado;
    AvisoCancelacion aviso = {0};
    if (!esperar_programa_limitado(pid, &estado, entrada, control, &aviso))
    {
        int error = errno;
        close(errores[0]);
        establecer_detalle(resultado, "Error al esperar el programa: %s.", strerror(error));
        return;
    }

    /* Solo se envia un errno: cabe en el pipe aun si esperamos antes de leer. */
    int error_ejecucion = 0;
    bool comunicado = leer_error_ejecucion(errores[0], &error_ejecucion);
    int error_comunicacion = errno;
    close(errores[0]);

    struct timespec fin;
    if (clock_gettime(CLOCK_MONOTONIC, &fin) == -1)
    {
        establecer_detalle(resultado, "Error al medir el tiempo del programa: %s.", strerror(errno));
        return;
    }

    resultado->tiempo_programa_segundos = diferencia_segundos(&inicio, &fin);

    if (aviso.motivo != AVISO_NINGUNO)
        aplicar_cancelacion(resultado, &aviso);
    else if (!comunicado)
        establecer_detalle(resultado, "Error al recibir el estado de ejecucion: %s.", strerror(error_comunicacion));
    else if (error_ejecucion != 0)
        establecer_detalle(resultado, "No se pudo lanzar el programa (errno %d): %s.", error_ejecucion, strerror(error_ejecucion));
    else if (WIFSIGNALED(estado))
        establecer_detalle(resultado, "El programa termino por la senal %d.", WTERMSIG(estado));
    else if (WIFEXITED(estado))
    {
        int codigo = WEXITSTATUS(estado);
        resultado->estado = codigo == 0 ? RESULTADO_COMPLETADO : RESULTADO_NO_COMPLETADO;
        establecer_detalle(resultado, "El programa termino con codigo de salida %d.", codigo);
    }
    else
        establecer_detalle(resultado, "El programa termino con un estado inesperado.");
}
```

Figura 15: Función `procesar_programa()`: espera, tiempo máximo y resultado final.

```c
/*
    Cierra canales del Meeseeks, prepara grupo, entrada y señales, y llama a execvp.
    Si falla envía errno por pipe y termina con _exit(127); no vuelve al Meeseeks.
 */
static void ejecutar_programa_hijo(char *const argumentos[], int errores, int respuesta, int cancelaciones)
{
    close(respuesta);
    close(cancelaciones);
    int error_ejecucion = setpgid(0, 0) == -1 ? errno : 0;

    /* El programa no debe consumir las siguientes opciones de la Box. */
    int entrada = open("/dev/null", O_RDONLY);
    if (entrada == -1)
        error_ejecucion = errno;
    else if (entrada != STDIN_FILENO)
    {
        if (dup2(entrada, STDIN_FILENO) == -1)
            error_ejecucion = errno;
        close(entrada);
    }

    sigset_t desbloquear;
    sigemptyset(&desbloquear);
    sigaddset(&desbloquear, SIGINT);
    sigaddset(&desbloquear, SIGTERM);
    if (sigprocmask(SIG_UNBLOCK, &desbloquear, NULL) == -1)
        error_ejecucion = errno;

    struct sigaction accion = {0};
    accion.sa_handler = SIG_DFL;
    sigemptyset(&accion.sa_mask);
    if (error_ejecucion == 0 && sigaction(SIGPIPE, &accion, NULL) == -1)
        error_ejecucion = errno;

    if (error_ejecucion == 0)
    {
        execvp(argumentos[0], argumentos);
        error_ejecucion = errno;
    }

    (void)escribir_completo(errores, &error_ejecucion, sizeof(error_ejecucion));
    close(errores);
    _exit(127);
}
```

Figura 16: Función `ejecutar_programa_hijo()`.

3.8. Sincronización y recursos compartidos

Los procesos comparten contadores relacionados con el total de Meeseeks creados, los activos y la profundidad máxima. Para ello, la Box prepara una zona de memoria compartida mediante `mmap()` y crea un semáforo POSIX con `sem_init()` configurado para compartirse entre procesos.

Antes de actualizar un contador, el proceso obtiene el semáforo mediante `sem_wait()`; después libera la sección crítica con `sem_post()`. El presupuesto de procesos se reserva antes de llamar a `fork()`, para impedir que varias ramas excedan simultáneamente el límite autorizado. Estos conceptos se relacionan con la sincronización y la memoria compartida explicadas en *Beginning Linux Programming* [1].

```c
/* Adquiere el semáforo compartido, reintentando si una señal interrumpe la espera. */
static bool bloquear_control(ControlCompartido *control)
{
    int estado;
    do
    {
        estado = sem_wait(&control->mutex);
    } while (estado == -1 && errno == EINTR);
    return estado == 0;
}

/*
    Reserva memoria compartida para los contadores y crea su semáforo con pshared=1.
    Las solicitudes no se comparten aquí, se transmiten por pipes.
 */
static ControlCompartido *crear_control(const Configuracion *configuracion, const struct timespec *inicio)
{
    /* /dev/zero permite mmap compartido sin depender de MAP_ANONYMOUS. */
    int descriptor = open("/dev/zero", O_RDWR);
    if (descriptor == -1)
        return NULL;
    ControlCompartido *control = mmap(NULL, sizeof(*control), PROT_READ | PROT_WRITE,
                                      MAP_SHARED, descriptor, 0);
    int error = errno;
    close(descriptor);
    if (control == MAP_FAILED)
    {
        errno = error;
        return NULL;
    }

    *control = (ControlCompartido){0};
    control->limites = *configuracion;
    control->inicio = *inicio;
    if (sem_init(&control->mutex, 1, 1) == -1)
    {
        error = errno;
        munmap(control, sizeof(*control));
        errno = error;
        return NULL;
    }
    return control;
}
```

Figura 17: Inicialización del control compartido y acceso protegido al semáforo.

```c
/* Reserva el grupo completo bajo el semáforo antes de fork, si quedan tiempo y cupos.reservado indica si alcanzó el presupuesto */
static bool reservar_meeseeks(ControlCompartido *control, unsigned int cantidad, bool *reservado)
{
    double restante = tiempo_disponible(control);
    if (restante < 0.0 || !bloquear_control(control))
        return false;

    /* Se reserva antes de fork */
    *reservado = restante > 0.0 && cantidad <= control->limites.max_procesos - control->creados;
    if (*reservado)
    {
        control->creados += cantidad;
        control->activos += cantidad;
    }
    return sem_post(&control->mutex) == 0;
}
```

Figura 18: Reserva de procesos en `reservar_meeseeks()`.

3.9. Contingencia y caos planetario

El programa establece un presupuesto predeterminado de 128 Meeseeks por solicitud y un plazo normal predeterminado de 300 segundos. Los parámetros pueden ajustarse para realizar pruebas controladas. Cuando no quedan cupos, no se permiten nuevas generaciones. De esta forma, una solicitud con facilidad cero no se resuelve ni provoca una creación ilimitada de procesos.

Si se supera el tiempo autorizado, se declara caos planetario y se comunica la cancelación por los pipes. Existe además un modo experimental que permite continuar más allá del plazo normal, pero con un límite extraordinario explícito y conservando el presupuesto de procesos.

```c
/* Calcula con CLOCK_MONOTONIC los segundos restantes; -1 indica un error del reloj. */
static double tiempo_disponible(const ControlCompartido *control)
{
    struct timespec ahora;

    if (clock_gettime(CLOCK_MONOTONIC, &ahora) == -1)
        return -1.0;

    double restante = plazo_efectivo(&control->limites) - diferencia_segundos(&control->inicio, &ahora);
    return fmax(0.0, restante);
}

/* Solicita una sola configuración para todas las solicitudes e inicializa la semilla. */
static bool configurar_box(Configuracion *configuracion)
{
    *configuracion = (Configuracion){.max_procesos = MAX_PROCESOS, .semilla = (unsigned int)time(NULL)};
    puts("\n--- CONFIGURACION INICIAL ---");

    if (!leer_plazo("Tiempo maximo para caos planetario en segundos", PLAZO_PREDETERMINADO, 0.0, &configuracion->plazo_segundos))
        return false;

    char *entrada = NULL;
    size_t capacidad = 0;
    while (1)
    {
        printf("Activar modo experimental? (s/n, Enter: n): ");
        fflush(stdout);
        if (getline(&entrada, &capacidad, stdin) == -1)
        {
            free(entrada);
            return false;
        }
        limpiar_espacios(entrada);
        configuracion->modo_experimental = strcmp(entrada, "s") == 0 || strcmp(entrada, "S") == 0;

        if (configuracion->modo_experimental || *entrada == '\0' || strcmp(entrada, "n") == 0 || strcmp(entrada, "N") == 0)
            break;

        puts("Ingrese s o n.");
    }
    free(entrada);
    return !configuracion->modo_experimental ||
           leer_plazo("Limite extraordinario en segundos (mayor que el normal)", LIMITE_EXTRAORDINARIO_PREDETERMINADO,
                      configuracion->plazo_segundos, &configuracion->limite_extraordinario);
}
```

Figura 19: Configuración del tiempo máximo y control del presupuesto.

3.10. Medición de tiempos y generación del informe

Se registran dos métricas diferentes para cada solicitud. El *tiempo transcurrido* se obtiene con `clock_gettime()` y el reloj `CLOCK_MONOTONIC`. Representa el intervalo desde el inicio de la atención hasta la terminación y recolección del árbol, incluidas las pausas simuladas. El *tiempo de CPU*, medido con `getrusage()`, acumula el trabajo de CPU de la Box y de los descendientes recogidos, sin contabilizar las pausas artificiales como tiempo de procesamiento.

Estas dos medidas no son equivalentes: el tiempo de CPU suma trabajo de procesos concurrentes y puede ser mayor que el tiempo transcurrido. Por ello, las comparaciones del apartado 0x02 utilizarán ambas medidas, indicarán su significado y evitarán interpretar el tiempo de CPU como una duración de reloj sin pausas. En el caso de programas externos se registra adicionalmente el tiempo transcurrido de dicho programa.

```c
/*
    Suma CPU de la Box y de sus descendientes ya recogidos: usuario y sistema.
    Las esperas no consumen CPU, la diferencia entre dos lecturas mide una solicitud.
 */
static double medir_cpu_total(void)
{
    struct rusage propio;
    struct rusage hijos;

    if (getrusage(RUSAGE_SELF, &propio) == -1 || getrusage(RUSAGE_CHILDREN, &hijos) == -1)
        return -1.0;

    double segundos = (double)propio.ru_utime.tv_sec + propio.ru_stime.tv_sec + hijos.ru_utime.tv_sec + hijos.ru_stime.tv_sec;

    double microsegundos = (double)propio.ru_utime.tv_usec + propio.ru_stime.tv_usec + hijos.ru_utime.tv_usec + hijos.ru_stime.tv_usec;

    return segundos + microsegundos / 1000000.0;
}
```

Figura 20: Función `medir_cpu_total()`.

```c
/* Prepara recursos y señales, atiende la solicitud, recoge estadísticas y mide su duración. */
static bool procesar_solicitud(RegistroSolicitud *registro, const Configuracion *configuracion)
{
    double cpu_inicio = medir_cpu_total();
    if (cpu_inicio < 0.0)
    {
        perror("Error al iniciar la medicion de CPU");
        return false;
    }

    struct timespec inicio;
    if (clock_gettime(CLOCK_MONOTONIC, &inicio) == -1)
    {
        perror("Error al iniciar la medicion de la solicitud");
        establecer_detalle(&registro->resultado, "No se pudo iniciar la medicion de la solicitud.");
        return false;
    }

    ControlCompartido *control = crear_control(configuracion, &inicio);
    if (control == NULL)
    {
        perror("Error al crear memoria compartida o semaforo");
        return false;
    }
    sigset_t bloqueo, anteriores;
    sigemptyset(&bloqueo);
    sigaddset(&bloqueo, SIGINT);
    sigaddset(&bloqueo, SIGTERM);
    if (sigprocmask(SIG_BLOCK, &bloqueo, &anteriores) == -1)
    {
        perror("Error al preparar cancelacion por senales");
        sem_destroy(&control->mutex);
        munmap(control, sizeof(*control));
        return false;
    }
    bool procesada = crear_meeseeks(&registro->solicitud, &registro->resultado, control);
    /* La Box consume las senales pendientes antes de restaurar su mascara. */
    while (consultar_interrupcion() > 0)
    {
    }
    if (sigprocmask(SIG_SETMASK, &anteriores, NULL) == -1)
    {
        perror("Error al restaurar la mascara de senales");
        procesada = false;
    }
    /* El hijo ya fue recogido: solo la Box destruye los recursos compartidos. */
    registro->resultado.meeseeks_creados = control->creados;
    registro->resultado.profundidad_maxima = control->profundidad_maxima;
    if (control->activos != 0)
```

Figura 21: Función `procesar_solicitud()`: preparación y medición inicial.

```
    if (control->activos != 0)
    {
        fputs("Box: contador de procesos activos inconsistente.\n", stderr);
        procesada = false;
    }
    if (sem_destroy(&control->mutex) == -1)
    {
        perror("Error al destruir el semaforo");
        procesada = false;
    }
    if (munmap(control, sizeof(*control)) == -1)
    {
        perror("Error al liberar memoria compartida");
        procesada = false;
    }

    struct timespec fin;
    if (clock_gettime(CLOCK_MONOTONIC, &fin) == -1)
    {
        perror("Error al terminar la medicion de la solicitud");
        return false;
    }

    registro->tiempo_transcurrido_segundos = diferencia_segundos(&inicio, &fin);
    double cpu_fin = medir_cpu_total();
    if (cpu_fin < 0.0)
    {
        perror("Error al terminar la medicion de CPU");
        return false;
    }
    registro->tiempo_cpu_segundos = cpu_fin - cpu_inicio;
    return procesada;
}
```

Figura 22: Función `procesar_solicitud()`: consolidación del resultado y estadísticas.

4. Resultados de ejecución: mínimo tres corridas por caso

Se realizaron tres corridas principales para cada uno de los casos del enunciado, además de pruebas complementarias para comprobar el manejo de solicitudes imposibles y errores. Las capturas muestran la ejecución de las solicitudes, la comunicación entre procesos y los resultados registrados por la Box.

4.1. Caso A: solicitudes textuales

4.1.1. Corrida A1: tarea fácil

Configuración utilizada: facilidad manual de 95, duración de simulación de 0.5 segundos, presupuesto de 128 Meeseeks y plazo normal de 300 segundos.

```
jose@jose:~/SO.T3-MrMeeseeks$ ./build/meeseeks
=== Box de Mr. Meeseeks ===

--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n): n
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 1

Solicitud: Quiero un cafe

--- CONFIGURACION DE TAREA TEXTUAL ---
Facilidad: 0 = imposible, 100 = muy facil.
1. Ingresar facilidad manualmente
2. Generar facilidad aleatoriamente
0. Cancelar solicitud
Opcion: 1
Facilidad (0 a 100): 95

Duracion de la simulacion:
1. Definida por el usuario
2. Aleatoria (entre 0.5 y 5 segundos)
0. Cancelar solicitud
Opcion: 1
Duracion en segundos (> 0): 0.5
Box: solicitud #1 recibida: "Quiero un cafe"
Box: tipo de solicitud: Texto (simulacion)
Box: facilidad: 95/100 (manual)
Box: duracion: 0.50 segundos (manual)
Box: pendiente de procesar.
```

Figura 23: Caso A, corrida 1: creación y atención de una tarea fácil.

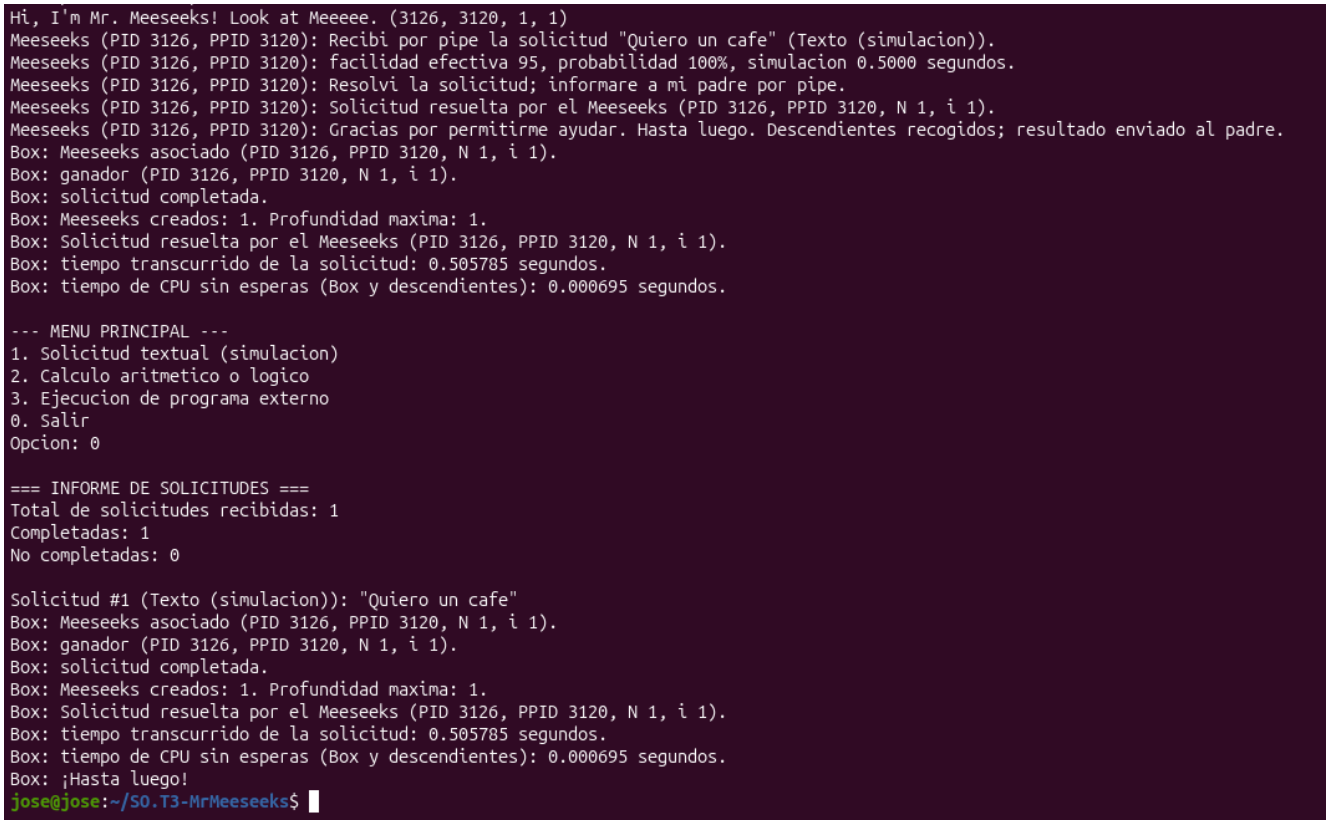


Figura 24: Caso A, corrida 1: resultado e informe final.

En esta primera corrida se ingresó la solicitud “Quiero un café”, con una facilidad de 95 y una duración de 0.5 segundos. Como la tarea tenía una probabilidad de éxito del 100 %, el primer Meeseeks (PID 3126) logró resolverla sin necesidad de crear hijos. El programa registró la solicitud como completada, con un solo Meeseeks creado y una profundidad máxima de 1. El tiempo transcurrido fue de 0.505785 segundos, mientras que el tiempo de CPU fue de apenas 0.000695 segundos, debido a que la mayor parte del tiempo correspondió a la espera simulada. Los resultados muestran que una tarea con facilidad alta puede resolverse con un único proceso, sin necesidad de solicitar ayuda a otros Meeseeks.

4.1.2. Corrida A2: tarea intermedia

Configuración utilizada: Se utilizó una facilidad de 65 y una duración de simulación de 0.5 segundos, para observar la creación de hijos y la comunicación del resultado entre los Meeseeks.

```
jose@jose:~/SO.T3-MrMeeseeks$ ./build/meeseeks
=== Box de Mr. Meeseeks ===

--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n): n
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 1

Solicitud: Quiero un caballo

--- CONFIGURACION DE TAREA TEXTUAL ---
Facilidad: 0 = imposible, 100 = muy facil.
1. Ingresar facilidad manualmente
2. Generar facilidad aleatoriamente
0. Cancelar solicitud
Opcion: 1
Facilidad (0 a 100): 65

Duracion de la simulacion:
1. Definida por el usuario
2. Aleatoria (entre 0.5 y 5 segundos)
0. Cancelar solicitud
Opcion: 1
Duracion en segundos (> 0): 0.5
Box: solicitud #1 recibida: "Quiero un caballo"
Box: tipo de solicitud: Texto (simulacion)
Box: facilidad: 65/100 (manual)
Box: duracion: 0.50 segundos (manual)
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (3612, 3607, 1, 1)
Meeseeks (PID 3612, PPID 3607): Recibi por pipe la solicitud "Quiero un caballo" (Texto (simulacion)).
Meeseeks (PID 3612, PPID 3607): facilidad efectiva 65, probabilidad 42.25%, simulacion 0.5000 segundos.
Meeseeks (PID 3612, PPID 3607): Necesito ayuda de 2 hijos.
```

Figura 25: Caso A, corrida 2: creación de hijos para una tarea intermedia.

```
Meeseeks (PID 3612, PPID 3607): Necesito ayuda de 2 hijos.
Meeseeks (PID 3612, PPID 3607): Cree hijo PID 3613 (N 2, i 1).
        Hi, I'm Mr. Meeseeks! Look at Meeeee. (3613, 3612, 2, 1)
        Meeseeks (PID 3613, PPID 3612): Recibi por pipe la solicitud "Quiero un caballo" (Texto (simulacion)).
        Meeseeks (PID 3613, PPID 3612): facilidad efectiva 72, probabilidad 51.84%, simulacion 0.5000 segundos.
Meeseeks (PID 3612, PPID 3607): Cree hijo PID 3614 (N 2, i 2).
        Hi, I'm Mr. Meeseeks! Look at Meeeee. (3614, 3612, 2, 2)
        Meeseeks (PID 3614, PPID 3612): Recibi por pipe la solicitud "Quiero un caballo" (Texto (simulacion)).
        Meeseeks (PID 3614, PPID 3612): facilidad efectiva 72, probabilidad 51.84%, simulacion 0.5000 segundos.
        Meeseeks (PID 3614, PPID 3612): Necesito ayuda de 2 hijos.
        Meeseeks (PID 3614, PPID 3612): Cree hijo PID 3615 (N 3, i 1).
        Meeseeks (PID 3614, PPID 3612): Cree hijo PID 3616 (N 3, i 2).
                Hi, I'm Mr. Meeseeks! Look at Meeeee. (3615, 3614, 3, 1)
                Meeseeks (PID 3615, PPID 3614): Recibi por pipe la solicitud "Quiero un caballo" (Texto (simulacion)).
                Meeseeks (PID 3615, PPID 3614): facilidad efectiva 77.6, probabilidad 60.2176%, simulacion 0.5000 segundos.
                Hi, I'm Mr. Meeseeks! Look at Meeeee. (3616, 3614, 3, 2)
                Meeseeks (PID 3616, PPID 3614): Recibi por pipe la solicitud "Quiero un caballo" (Texto (simulacion)).
                Meeseeks (PID 3616, PPID 3614): facilidad efectiva 77.6, probabilidad 60.2176%, simulacion 0.5000 segundos.
        Meeseeks (PID 3613, PPID 3612): Resolvi la solicitud; informare a mi padre por pipe.
        Meeseeks (PID 3613, PPID 3612): Solicitud resuelta por el Meeseeks (PID 3613, PPID 3612, N 2, i 1).
        Meeseeks (PID 3613, PPID 3612): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Meeseeks (PID 3612, PPID 3607): Recogi con waitpid al hijo PID 3613.
Meeseeks (PID 3612, PPID 3607): Hijo informo por pipe: gano PID 3613. Aviso a sus hermanos.
Meeseeks (PID 3612, PPID 3607): Aviso por pipe a hijo PID 3614: gano PID 3613; deten tu subarbol.
        Meeseeks (PID 3614, PPID 3612): Recibi cancelacion de mi padre; aviso a mis descendientes.
                Meeseeks (PID 3615, PPID 3614): Trabajo cancelado: gano el Meeseeks PID 3613.
                Meeseeks (PID 3615, PPID 3614): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
        Meeseeks (PID 3614, PPID 3612): Aviso por pipe a hijo PID 3615: gano PID 3613; deten tu subarbol.
        Meeseeks (PID 3614, PPID 3612): Aviso por pipe a hijo PID 3616: gano PID 3613; deten tu subarbol.
                Meeseeks (PID 3616, PPID 3614): Trabajo cancelado: gano el Meeseeks PID 3613.
                Meeseeks (PID 3616, PPID 3614): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
        Meeseeks (PID 3614, PPID 3612): Recogi con waitpid al hijo PID 3615.
        Meeseeks (PID 3614, PPID 3612): Recogi con waitpid al hijo PID 3616.
        Meeseeks (PID 3614, PPID 3612): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Meeseeks (PID 3612, PPID 3607): Recogi con waitpid al hijo PID 3614.
Meeseeks (PID 3612, PPID 3607): Solicitud resuelta por el Meeseeks (PID 3613, PPID 3612, N 2, i 1).
Meeseeks (PID 3612, PPID 3607): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
```

Figura 26: Caso A, corrida 2: ganador, notificación y cancelación de hermanos.

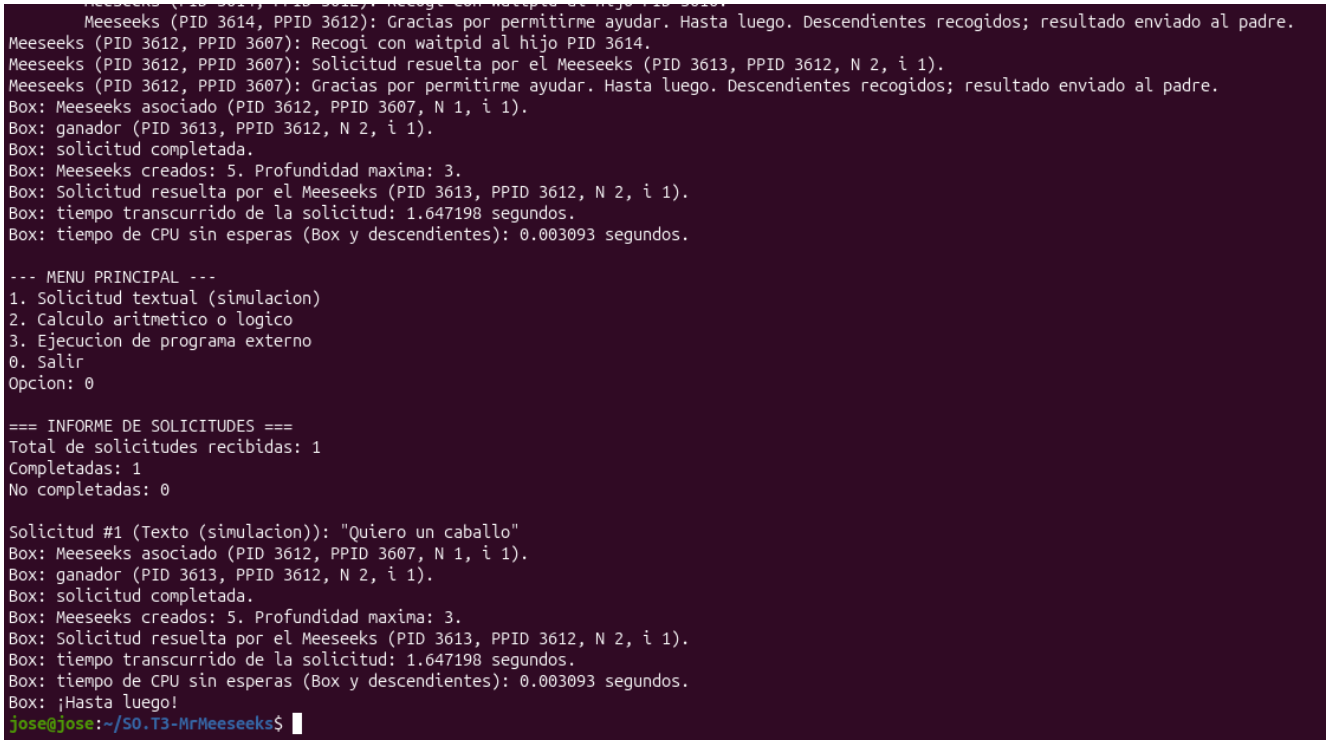


Figura 27: Caso A, corrida 2: informe de la atención intermedia.

En esta segunda corrida se ingresó la solicitud “Quiero un caballo”, con una facilidad de 65 y una duración de 0.5 segundos. El primer Meeseeks (PID 3612) no logró resolverla, por lo que solicitó ayuda y creó dos hijos. Uno de ellos también necesitó ayuda y creó otros dos Meeseeks, alcanzando un total de cinco procesos y una profundidad máxima de tres niveles. Finalmente, el Meeseeks con PID 3613, perteneciente al segundo nivel, resolvió la solicitud y comunicó el resultado a su padre mediante un pipe. Este avisó al otro hijo, que canceló su trabajo y el de sus descendientes. El tiempo transcurrido fue de 1.647198 segundos y el tiempo de CPU fue de 0.003093 segundos. Con esta prueba se observó cómo una tarea intermedia puede necesitar varios procesos y cómo la solución se comunica entre ellos para detener el trabajo que ya no es necesario.

4.1.3. Corrida A3: tarea difícil

Configuración utilizada: Se utilizó una facilidad de 25, una duración de simulación de 0.5 segundos y un presupuesto máximo de 128 Meeseeks. La corrida permitió observar la creación de cinco hijos y la formación de nuevas generaciones.

```
jose@jose:~/SO.T3-MrMeeseeks$ ./build/meeseeks
=== Box de Mr. Meeseeks ===

--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n): n
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 1

Solicitud: Me gustaria volar

--- CONFIGURACION DE TAREA TEXTUAL ---
Facilidad: 0 = imposible, 100 = muy facil.
1. Ingresar facilidad manualmente
2. Generar facilidad aleatoriamente
0. Cancelar solicitud
Opcion: 1
Facilidad (0 a 100): 25

Duracion de la simulacion:
1. Definida por el usuario
2. Aleatoria (entre 0.5 y 5 segundos)
0. Cancelar solicitud
Opcion: 1
Duracion en segundos (> 0): 0.5
Box: solicitud #1 recibida: "Me gustaria volar"
Box: tipo de solicitud: Texto (simulacion)
Box: facilidad: 25/100 (manual)
Box: duracion: 0.50 segundos (manual)
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (3754, 3752, 1, 1)
Meeseeks (PID 3754, PPID 3752): Recibi por pipe la solicitud "Me gustaria volar" (Texto (simulacion)).
Meeseeks (PID 3754, PPID 3752): facilidad efectiva 25, probabilidad 6.25%, simulacion 0.5000 segundos.
Meeseeks (PID 3754, PPID 3752): Necesito ayuda de 5 hijos.
Meeseeks (PID 3754, PPID 3752): Cree hijo PID 3755 (N 2, i 1).
        Hi, I'm Mr. Meeseeks! Look at Meeeee. (3755, 3754, 2, 1)
        Meeseeks (PID 3755, PPID 3754): Recibi por pipe la solicitud "Me gustaria volar" (Texto (simulacion)).
Meeseeks (PID 3754, PPID 3752): Cree hijo PID 3756 (N 2, i 2).
        Meeseeks (PID 3755, PPID 3754): facilidad efectiva 40, probabilidad 16%, simulacion 0.5000 segundos.
Meeseeks (PID 3754, PPID 3752): Cree hijo PID 3757 (N 2, i 3).
        Hi, I'm Mr. Meeseeks! Look at Meeeee. (3757, 3754, 2, 3)
        Meeseeks (PID 3757, PPID 3754): Recibi por pipe la solicitud "Me gustaria volar" (Texto (simulacion)).
        Meeseeks (PID 3757, PPID 3754): facilidad efectiva 40, probabilidad 16%, simulacion 0.5000 segundos.
Meeseeks (PID 3754, PPID 3752): Cree hijo PID 3758 (N 2, i 4).
```

Figura 28: Caso A, corrida 3: primer nivel y creación de cinco hijos.

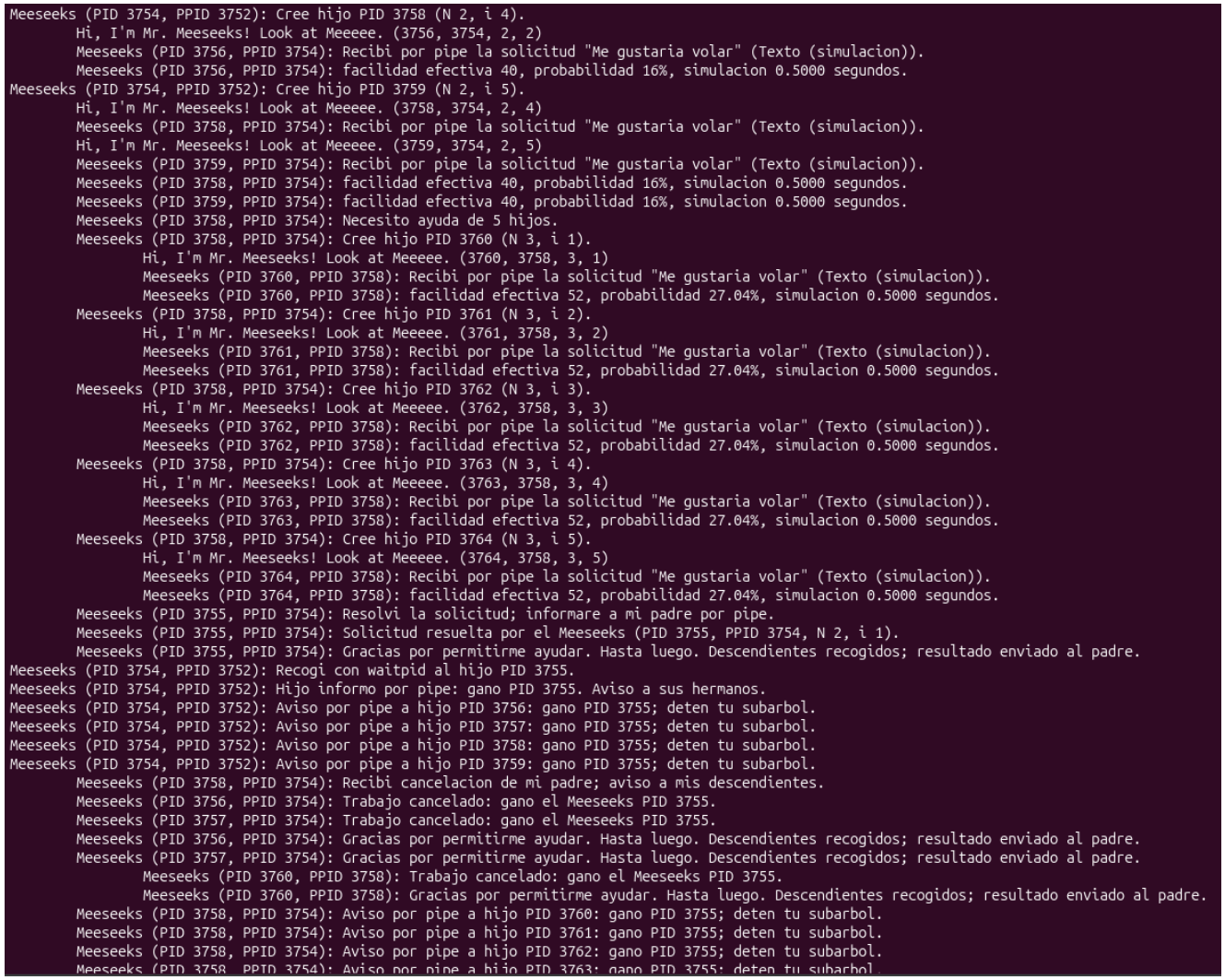


Figura 29: Caso A, corrida 3: generaciones y comunicación en cascada.

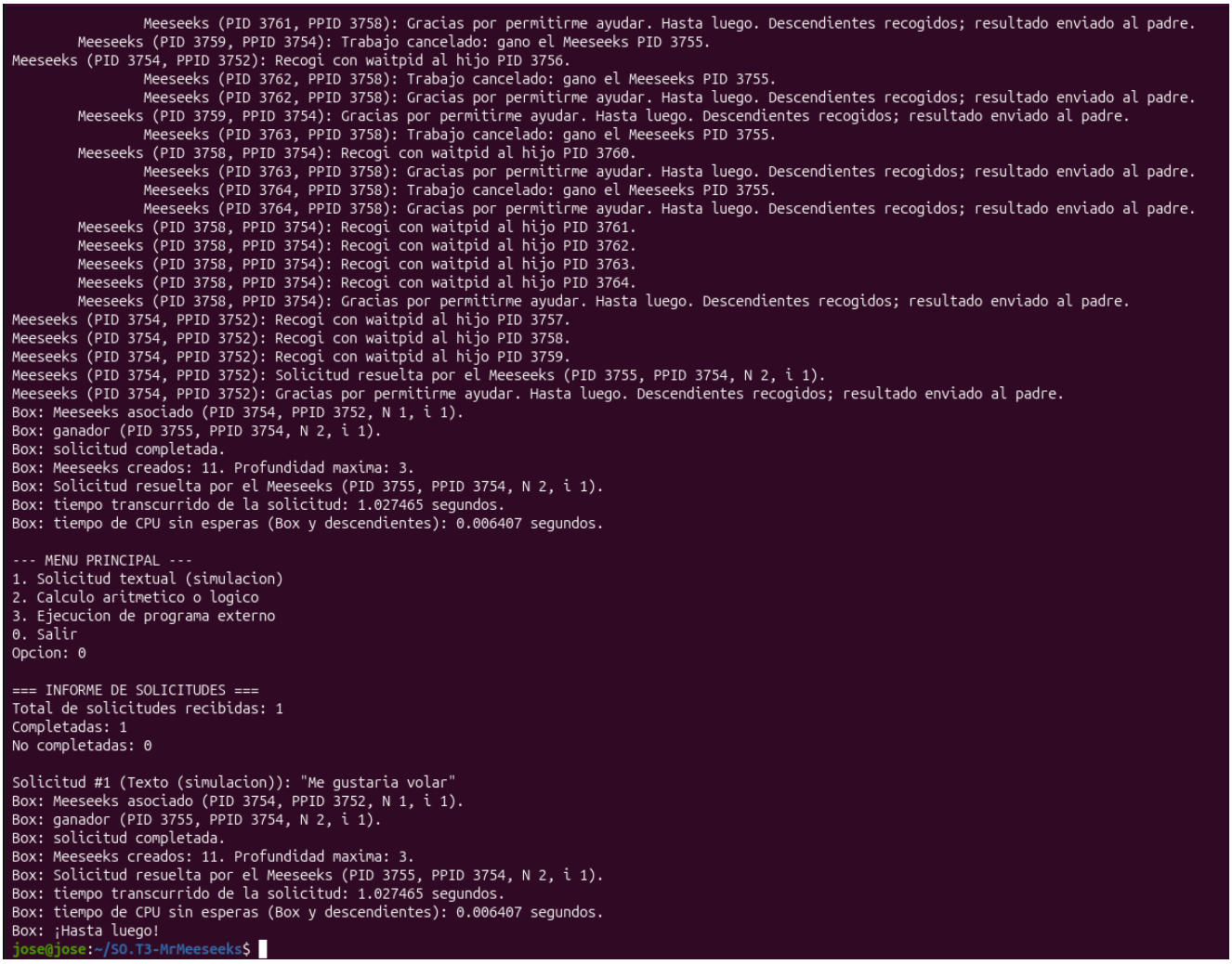


Figura 30: Caso A, corrida 3: estadísticas de la tarea difícil.

En esta tercera corrida se ingresó la solicitud “Me gustaría volar”, con una facilidad de 25 y una duración de 0.5 segundos. El primer Meeseeks (PID 3754) tenía una probabilidad de éxito del 6.25 % y, al no lograr resolver la tarea, creó cinco hijos para solicitar ayuda. Uno de ellos también necesitó ayuda y generó cinco Meeseeks adicionales, alcanzando un total de 11 procesos y una profundidad máxima de tres niveles. Finalmente, el Meeseeks con PID 3755, perteneciente al segundo nivel, logró resolver la solicitud y comunicó el resultado a su padre mediante un pipe. Este avisó a los demás hijos para que cancelaran su trabajo y detuvieran a sus descendientes. La solicitud se completó en 1.027465 segundos, con un tiempo de CPU de 0.006407 segundos. En comparación con las corridas anteriores, esta tarea necesitó más procesos debido a su menor facilidad. Además, se pudo observar cómo los Meeseeks trabajan al mismo tiempo y cómo la solución permite detener las demás ramas del árbol.

4.1.4. Corrida extra A4: solicitud imposible

Configuración utilizada: Se utilizó una facilidad de 0, una duración de simulación de 0.5 segundos y un presupuesto máximo de 128 Meeseeks. Esta prueba permitió comprobar que una solicitud imposible no puede resolverse y que la creación de procesos se detiene cuando se agota el presupuesto disponible.

```
jose@jose:~/SO.T3-MrMeeseeks$ ./build/meeseeks
=== Box de Mr. Meeseeks ===

--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n): n
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 1

Solicitud: Quiero teletransportarme

--- CONFIGURACION DE TAREA TEXTUAL ---
Facilidad: 0 = imposible, 100 = muy facil.
1. Ingresar facilidad manualmente
2. Generar facilidad aleatoriamente
0. Cancelar solicitud
Opcion: 1
Facilidad (0 a 100): 0

Duracion de la simulacion:
1. Definida por el usuario
2. Aleatoria (entre 0.5 y 5 segundos)
0. Cancelar solicitud
Opcion: 1
Duracion en segundos (> 0): 0.5
Box: solicitud #1 recibida: "Quiero teletransportarme"
Box: tipo de solicitud: Texto (simulacion)
Box: facilidad: 0/100 (manual)
Box: duracion: 0.50 segundos (manual)
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (3797, 3791, 1, 1)
Meeseeks (PID 3797, PPID 3791): Recibi por pipe la solicitud "Quiero teletransportarme" (Texto (simulacion)).
Meeseeks (PID 3797, PPID 3791): facilidad efectiva 0, probabilidad 0%, simulacion 0.5000 segundos.
```

Figura 31: Contingencia para una solicitud imposible.

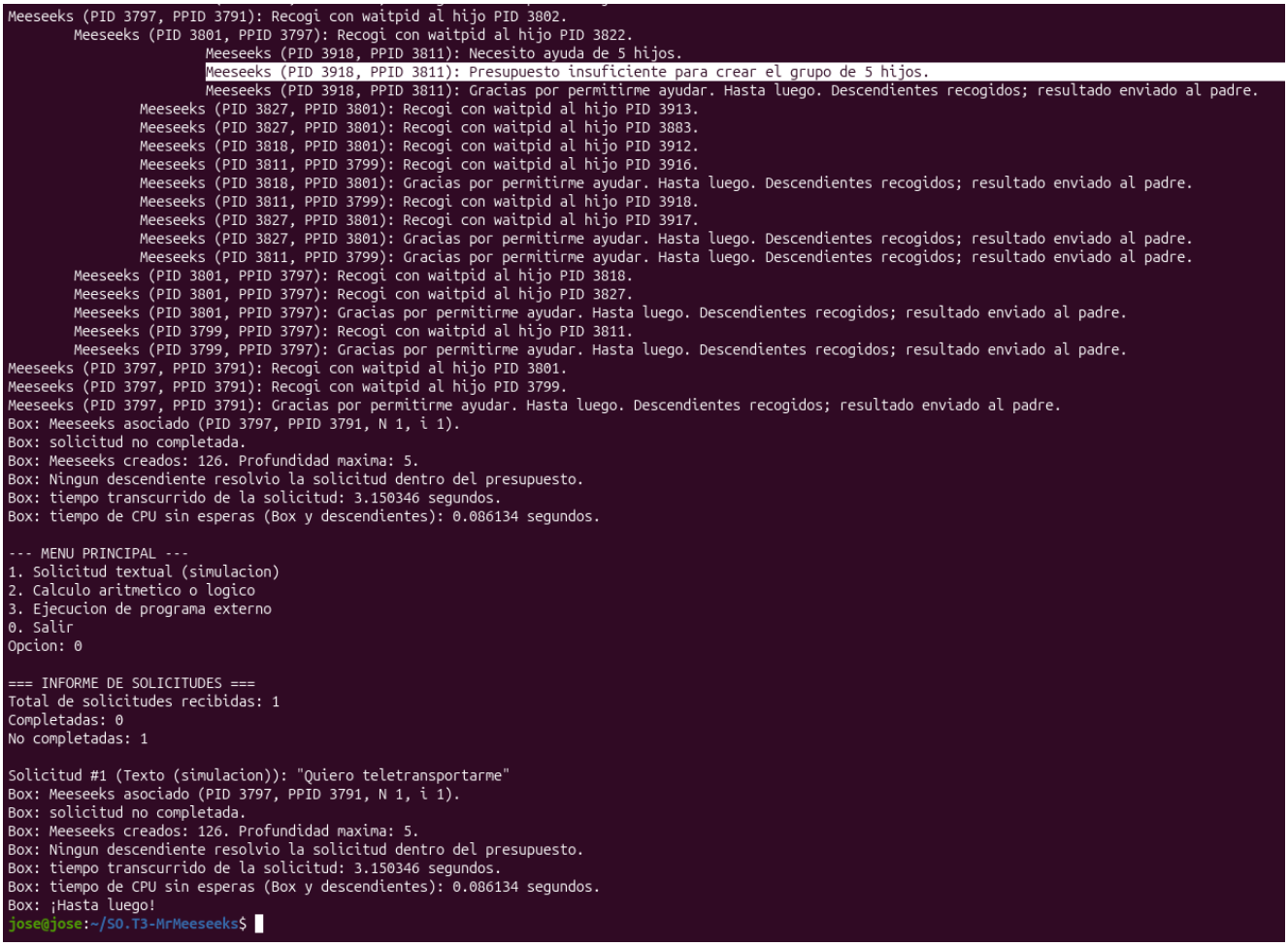


Figura 32: Informe de la solicitud imposible.

En esta prueba se ingresó la solicitud “Quiero teletransportarme”, con una facilidad de 0 y una duración de 0.5 segundos. Al tratarse de una tarea imposible, todos los Meeseeks mantuvieron una probabilidad de éxito del 0 %, por lo que continuaron solicitando ayuda y creando nuevos procesos. Durante la ejecución se alcanzó un total de 126 Meeseeks y una profundidad máxima de cinco niveles. Cuando el presupuesto disponible ya no permitió crear grupos completos de cinco hijos, los procesos comenzaron a finalizar sin resolver la solicitud. El tiempo transcurrido fue de 3.150346 segundos, mientras que el tiempo de CPU fue de 0.086134 segundos. Finalmente, la Box registró la solicitud como no completada. Esta prueba permitió comprobar que las tareas con facilidad 0 nunca se resuelven y que el límite de procesos evita que los Meeseeks continúen multiplicándose indefinidamente.

4.2. Caso B: operaciones aritméticas y lógicas

4.2.1. Corrida B1: expresión aritmética

Entrada utilizada: `2 + 3 * 4`. Se comprueba la prioridad de operadores. El resultado matemático esperado es 14.

```
jose@jose:~/SO.T3-MrMeeseeks$ ./build/meeseeks
=== Box de Mr. Meeseeks ===

--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n): n
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 2

Solicitud: 2 + 3 * 4
Box: solicitud #1 recibida: "2 + 3 * 4"
Box: tipo de solicitud: Calculo aritmetico o logico
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (4395, 4390, 1, 1)
Meeseeks (PID 4395, PPID 4390): Recibi por pipe la solicitud "2 + 3 * 4" (Calculo aritmetico o logico).
Meeseeks (PID 4395, PPID 4390): Resultado: 14
Meeseeks (PID 4395, PPID 4390): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 4395, PPID 4390, N 1, i 1).
Box: ganador (PID 4395, PPID 4390, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Resultado: 14
Box: tiempo transcurrido de la solicitud: 0.000507 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.000552 segundos.
```

Figura 33: Caso B, corrida 1: expresión aritmética y Meeseeks.

```
Box: tiempo de CPU sin esperas (Box y descendientes): 0.000552 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 0

=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 1
Completadas: 1
No completadas: 0

Solicitud #1 (Calculo aritmetico o logico): "2 + 3 * 4"
Box: Meeseeks asociado (PID 4395, PPID 4390, N 1, i 1).
Box: ganador (PID 4395, PPID 4390, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Resultado: 14
Box: tiempo transcurrido de la solicitud: 0.000507 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.000552 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 34: Caso B, corrida 1: resultado e informe.

En esta primera corrida del caso B se ingresó la operación “2 + 3 * 4”, con el objetivo de comprobar que el programa respeta la prioridad de los operadores matemáticos. El Meeseeks creado (PID 4395) recibió la expresión mediante un pipe y obtuvo como resultado 14, realizando primero la multiplicación y después la suma. La solicitud se registró como completada, utilizando un solo Meeseeks y alcanzando una profundidad máxima de 1. El tiempo transcurrido fue de 0.000507 segundos y el tiempo de CPU fue de 0.000552 segundos. Estos valores muestran que la operación se resolvió rápidamente y sin necesidad de crear procesos adicionales. Con esta prueba se comprobó que el programa puede interpretar correctamente una expresión aritmética y comunicar el resultado a la Box.

4.2.2. Corrida B2: expresión lógica

Entrada utilizada: `(5 > 2) && (2 < 4)`. Ambas comparaciones son verdaderas, por lo que el valor lógico esperado es 1.

```
jose@jose:~/SO.T3-MrMeeseeks$ ./build/meeseeks
=== Box de Mr. Meeseeks ===

--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n): n
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 2

Solicitud: (5 > 2) && (2 < 4)
Box: solicitud #1 recibida: "(5 > 2) && (2 < 4)"
Box: tipo de solicitud: Calculo aritmetico o logico
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (2963, 2957, 1, 1)
Meeseeks (PID 2963, PPID 2957): Recibi por pipe la solicitud "(5 > 2) && (2 < 4)" (Calculo aritmetico o logico).
Meeseeks (PID 2963, PPID 2957): Resultado: 1
Meeseeks (PID 2963, PPID 2957): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 2963, PPID 2957, N 1, i 1).
Box: ganador (PID 2963, PPID 2957, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Resultado: 1
Box: tiempo transcurrido de la solicitud: 0.000552 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.000645 segundos.
```

Figura 35: Caso B, corrida 2: expresión lógica.

```
=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 2
Completadas: 1
No completadas: 1

Solicitud #1 (Calculo aritmetico o logico): "(5 > 2) && (2 < 4)"
Box: Meeseeks asociado (PID 2963, PPID 2957, N 1, i 1).
Box: ganador (PID 2963, PPID 2957, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Resultado: 1
Box: tiempo transcurrido de la solicitud: 0.000552 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.000645 segundos.

Solicitud #2 (Calculo aritmetico o logico): "5 + * 2"
Box: Meeseeks asociado (PID 2964, PPID 2957, N 1, i 1).
Box: solicitud no completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Expresion invalida cerca de la posicion 5 (contada desde 1).
Box: tiempo transcurrido de la solicitud: 0.000598 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.000650 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 36: Caso B, corrida 2 y 3: resultado e informe de error.

En esta segunda corrida se ingresó la expresión lógica `(5 > 2) && (2 < 4)`, que contiene dos comparaciones unidas mediante el operador AND. El Meeseeks (PID 2963) recibió la solicitud por medio de un pipe y obtuvo como resultado 1, debido a que ambas condiciones eran verdaderas. La solicitud se completó utilizando un solo Meeseeks, con una profundidad máxima de 1. El tiempo transcurrido fue de 0.000552 segundos y el tiempo de CPU de 0.000645 segundos. Con esta prueba se comprobó que el programa también puede interpretar expresiones lógicas y devolver correctamente su resultado a la Box.

4.2.3. Corrida B3: expresión inválida

Entrada utilizada: `5 + * 2`. Se observó un error de sintaxis, y la solicitud se registró como no completada. También se documentó por separado una prueba complementaria de división por cero.

```
--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 2

Solicitud: 5 + * 2
Box: solicitud #2 recibida: "5 + * 2"
Box: tipo de solicitud: Calculo aritmetico o logico
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (2964, 2957, 1, 1)
Meeseeks (PID 2964, PPID 2957): Recibi por pipe la solicitud "5 + * 2" (Calculo aritmetico o logico).
Meeseeks (PID 2964, PPID 2957): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 2964, PPID 2957, N 1, i 1).
Box: solicitud no completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Expresion invalida cerca de la posicion 5 (contada desde 1).
Box: tiempo transcurrido de la solicitud: 0.000598 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.000650 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 0
```

Figura 37: Caso B, corrida 3: expresión inválida.

En esta tercera corrida se ingresó la expresión `5 + * 2`, con el propósito de comprobar cómo responde el programa ante una operación incorrecta. El Meeseeks (PID 2964) recibió la solicitud y detectó un error de sintaxis cerca de la posición 5, debido a que los operadores no estaban colocados correctamente. La solicitud se registró como no completada, utilizando un solo Meeseeks y alcanzando una profundidad máxima de 1. El tiempo transcurrido fue de 0.000598 segundos y el tiempo de CPU de 0.000650 segundos. Como se muestra en la Figura 36, el informe final registró las dos solicitudes realizadas, de las cuales una se completó correctamente y la otra presentó un error. Esto permitió comprobar que el programa identifica expresiones inválidas sin interrumpir su funcionamiento ni afectar el registro de las demás solicitudes.

```
--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 2

Solicitud: 10 / 0
Box: solicitud #1 recibida: "10 / 0"
Box: tipo de solicitud: Calculo aritmetico o logico
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (3038, 3036, 1, 1)
Meeseeks (PID 3038, PPID 3036): Recibi por pipe la solicitud "10 / 0" (Calculo aritmetico o logico).
Meeseeks (PID 3038, PPID 3036): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 3038, PPID 3036, N 1, i 1).
Box: solicitud no completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Division por cero.
Box: tiempo transcurrido de la solicitud: 0.000720 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.000805 segundos.
```

Figura 38: Prueba complementaria B4: división por cero.

4.3. Caso C: ejecución de programas externos

4.3.1. Corrida C1: programa exitoso

Entrada utilizada: `pwd`. La salida contiene el directorio actual y el programa normalmente retorna código 0.

```
jose@jose:~/SO.T3-MrMeeseeks$ ./build/meeseeks
=== Box de Mr. Meeseeks ===

--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n): n
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 3

Solicitud: pwd
Box: solicitud #1 recibida: "pwd"
Box: tipo de solicitud: Programa externo
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (3058, 3052, 1, 1)
Meeseeks (PID 3058, PPID 3052): Recibi por pipe la solicitud "pwd" (Programa externo).
/home/jose/SO.T3-MrMeeseeks
Meeseeks (PID 3058, PPID 3052): El programa termino con codigo de salida 0.
Meeseeks (PID 3058, PPID 3052): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 3058, PPID 3052, N 1, i 1).
Box: ganador (PID 3058, PPID 3052, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: El programa termino con codigo de salida 0.
Box: tiempo transcurrido de la solicitud: 0.449258 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.005700 segundos.
Box: tiempo del programa externo: 0.448497 segundos.
```

Figura 39: Caso C, corrida 1: ejecución de `pwd`.

```
Box: tiempo de CPU sin esperas (Box y descendientes): 0.005700 segundos.
Box: tiempo del programa externo: 0.448497 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 0

=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 1
Completadas: 1
No completadas: 0

Solicitud #1 (Programa externo): "pwd"
Box: Meeseeks asociado (PID 3058, PPID 3052, N 1, i 1).
Box: ganador (PID 3058, PPID 3052, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: El programa termino con codigo de salida 0.
Box: tiempo transcurrido de la solicitud: 0.449258 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.005700 segundos.
Box: tiempo del programa externo: 0.448497 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 40: Caso C, corrida 1: estado del programa e informe.

En esta primera corrida del caso C se ingresó el comando `pwd`, que permite mostrar el directorio actual del sistema. El Meeseeks (PID 3058) recibió la solicitud mediante un pipe y ejecutó el programa, mostrando la ruta `/home/jose/SO.T3-MrMeeseeks`. La ejecución terminó con un código de salida 0, por lo que la Box registró la solicitud como completada. Se utilizó un solo Meeseeks y se alcanzó una profundidad máxima de 1. El tiempo transcurrido fue de 0.449258 segundos, el tiempo de CPU fue de 0.005700 segundos y la duración del programa externo fue de 0.448497 segundos. Con esta prueba se comprobó que los Meeseeks pueden ejecutar programas externos de Linux y comunicar correctamente su estado de terminación a la Box.

4.3.2. Corrida C2: programa que retorna error

Entrada utilizada: `ls /directorio6767`. El programa se ejecutó, pero terminó con código de salida 2 porque el directorio indicado no existía.

```
jose@jose:~/SO.T3-MrMeeseeks$ ./build/meeseeks
=== Box de Mr. Meeseeks ===

--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n):
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 3

Solicitud: ls /directorio6767
Box: solicitud #1 recibida: "ls /directorio6767"
Box: tipo de solicitud: Programa externo
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (3072, 3067, 1, 1)
Meeseeks (PID 3072, PPID 3067): Recibi por pipe la solicitud "ls /directorio6767" (Programa externo).
ls: no se puede acceder a ''/directorio6767'': No existe el archivo o directorio
Meeseeks (PID 3072, PPID 3067): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 3072, PPID 3067, N 1, i 1).
Box: solicitud no completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: El programa termino con codigo de salida 2.
Box: tiempo transcurrido de la solicitud: 0.011520 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.007869 segundos.
Box: tiempo del programa externo: 0.010365 segundos.
```

Figura 41: Caso C, corrida 2: programa con código de error.

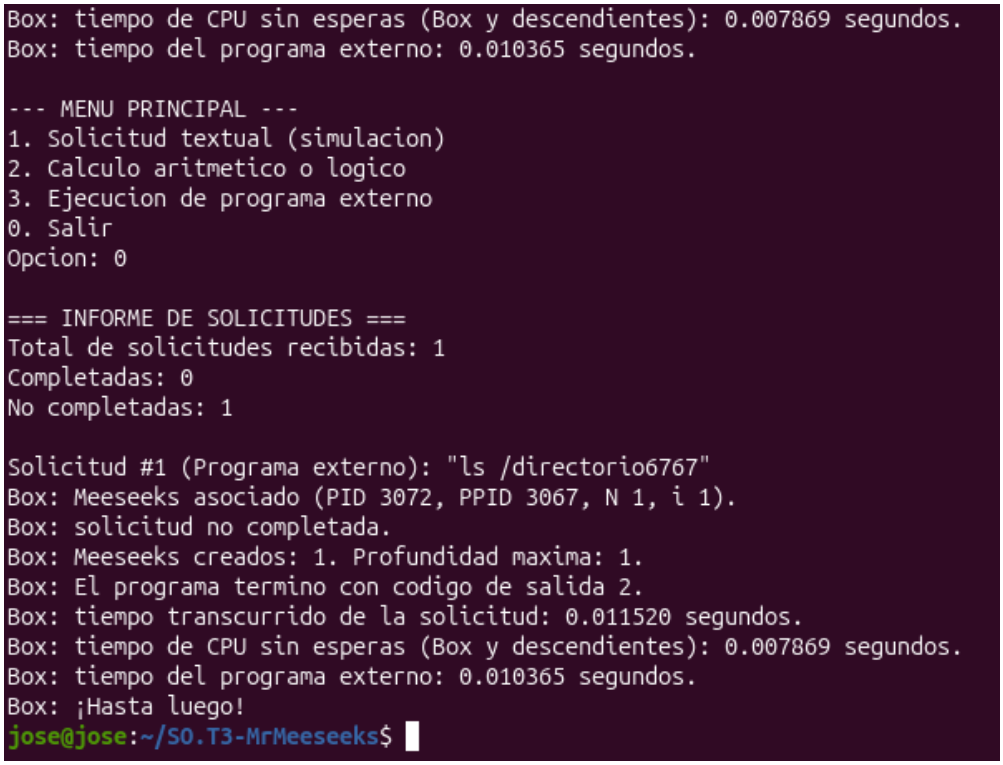


Figura 42: Caso C, corrida 2: informe de programa fallido.

En esta segunda corrida del caso C se ingresó el comando `ls /directorio6767`, con el propósito de comprobar cómo responde el programa cuando una ejecución externa presenta un error. El Meeseeks (PID 3072) recibió la solicitud mediante un pipe e intentó acceder al directorio indicado, pero este no existía. Por esta razón, el programa terminó con un código de salida 2 y la Box registró la solicitud como no completada. Se utilizó un solo Meeseeks, con una profundidad máxima de 1. El tiempo transcurrido fue de 0.011520 segundos, el tiempo de CPU fue de 0.007869 segundos y la duración del programa externo fue de 0.010365 segundos. Esta prueba permitió comprobar que la Box puede identificar cuando un programa externo falla y registrar correctamente su resultado, sin interrumpir su funcionamiento.

4.3.3. Corrida C3: programa interrumpido por plazo

Entrada utilizada: `sleep 10`, con un plazo normal de cinco segundos. Se activó la cancelación por caos planetario y la solicitud se registró como no completada.

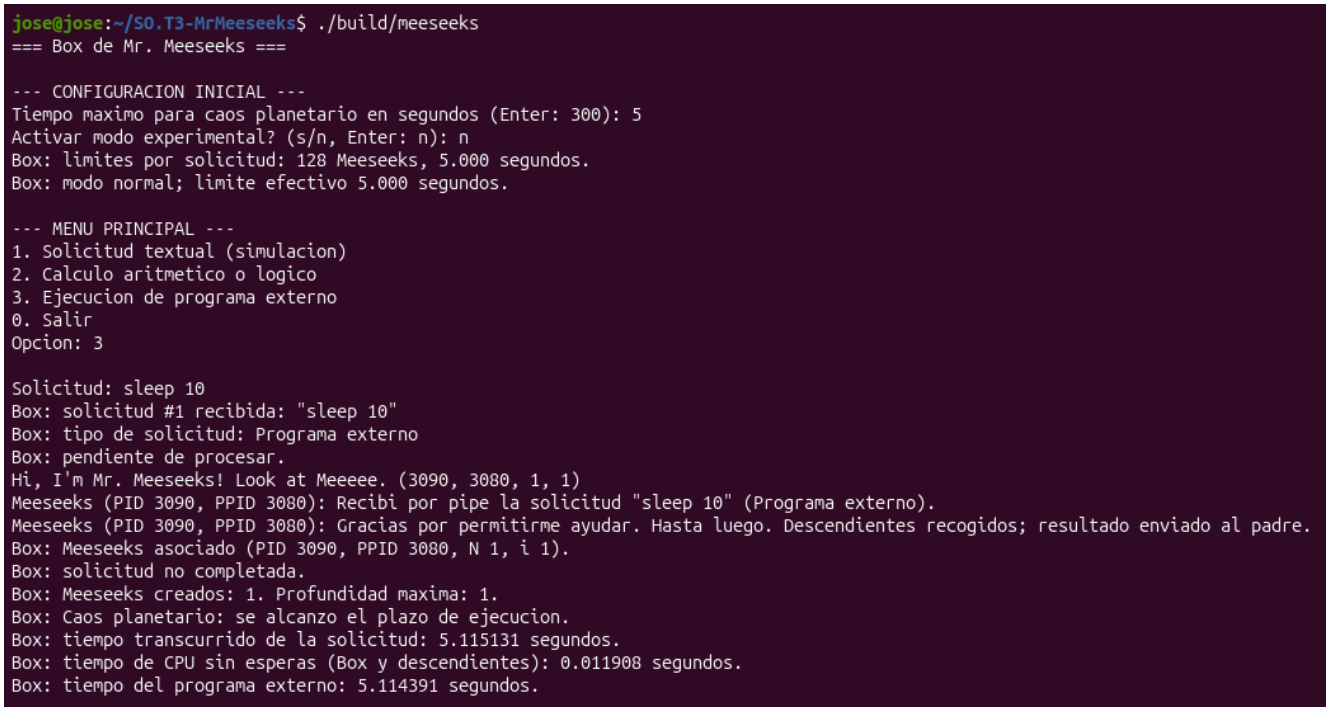


Figura 43: Caso C, corrida 3: ejecución y cancelación por plazo.

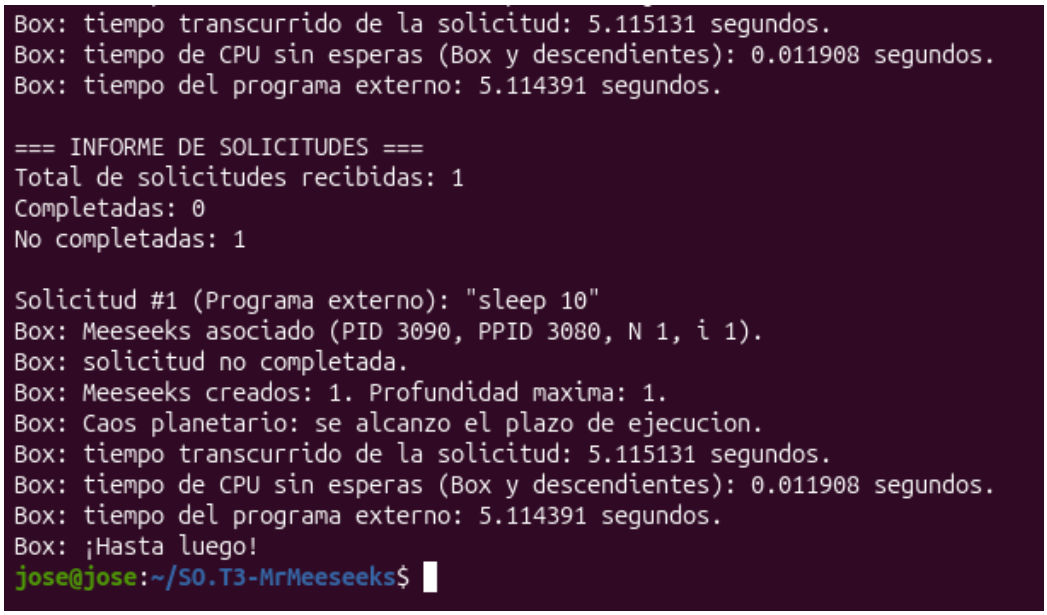


Figura 44: Caso C, corrida 3: informe del caos planetario.

En esta tercera corrida del caso C se ejecutó el comando `sleep 10`, estableciendo un tiempo máximo de 5 segundos para comprobar el funcionamiento del caos planetario. El Meeseeks (PID 3090) recibió la solicitud y comenzó la ejecución del programa externo, pero al superarse el plazo establecido, la Box inició su cancelación y registró la solicitud como no completada. Se utilizó un solo Meeseeks, con una profundidad máxima de 1. El tiempo transcurrido fue de 5.115131 segundos, el tiempo de CPU fue de 0.011908 segundos y la duración del programa externo fue de 5.114391 segundos. Estos resultados muestran que el programa puede controlar el tiempo de ejecución de una solicitud y detenerla cuando supera el límite permitido, evitando que continúe ejecutándose indefinidamente.

```
--- CONFIGURACION INICIAL ---
Tiempo maximo para caos planetario en segundos (Enter: 300):
Activar modo experimental? (s/n, Enter: n):
Box: limites por solicitud: 128 Meeseeks, 300.000 segundos.
Box: modo normal; limite efectivo 300.000 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 3

Solicitud: progama_inexistente_xyz
Box: solicitud #1 recibida: "progama_inexistente_xyz"
Box: tipo de solicitud: Programa externo
Box: pendiente de procesar.
Hi, I'm Mr. Meeseeks! Look at Meeeee. (3102, 3097, 1, 1)
Meeseeks (PID 3102, PPID 3097): Recibi por pipe la solicitud "progama_inexistente_xyz" (Programa externo).
Meeseeks (PID 3102, PPID 3097): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 3102, PPID 3097, N 1, i 1).
Box: solicitud no completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: No se pudo lanzar el programa (errno 2): No such file or directory.
Box: tiempo transcurrido de la solicitud: 0.012137 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.002211 segundos.
Box: tiempo del programa externo: 0.010888 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 0

=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 1
Completadas: 0
No completadas: 1

Solicitud #1 (Programa externo): "progama_inexistente_xyz"
Box: Meeseeks asociado (PID 3102, PPID 3097, N 1, i 1).
Box: solicitud no completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: No se pudo lanzar el programa (errno 2): No such file or directory.
Box: tiempo transcurrido de la solicitud: 0.012137 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.002211 segundos.
Box: tiempo del programa externo: 0.010888 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 45: Prueba complementaria C4: programa inexistente.

5. Experimentos comparativos y análisis (0x02)

En esta sección se comparan los resultados obtenidos durante las pruebas del programa. Se analizaron las diferencias en la cantidad de Meeseeks creados, la profundidad de los árboles, los tiempos de ejecución y los mecanismos de contingencia. Para ello, se utilizaron las mediciones registradas en las corridas y en los experimentos realizados.

5.1. Experimento E1: influencia de la facilidad

Pregunta de investigación: ¿Cómo afecta la facilidad inicial de una solicitud a la cantidad de Meeseeks creados, la profundidad del árbol y los tiempos de ejecución?

Diseño experimental: Se realizaron tres corridas para cada facilidad: 95, 65, 25 y 0, obteniendo un total de doce ejecuciones. Se utilizó una duración de simulación de 0.5 segundos, un plazo máximo de 300 segundos y un presupuesto de 128 Meeseeks por solicitud. Se mantuvo desactivado el modo experimental. Aunque los textos de las solicitudes variaron, la simulación utiliza la facilidad para determinar la probabilidad de éxito.

Cuadro 3: Comparación de solicitudes textuales según la facilidad.

Facilidad	Corrida	Creados	Prof.	Éxito	Tiempo (s)	CPU (s)
95	1	1	1	Sí	0.505295	0.001270
95	2	1	1	Sí	0.746402	0.001182
95	3	1	1	Sí	0.504778	0.001013
65	1	3	2	Sí	1.010167	0.002321
65	2	11	5	Sí	2.048549	0.007845
65	3	1	1	Sí	0.512464	0.001000
25	1	6	2	Sí	1.015842	0.004043
25	2	61	4	Sí	2.454516	0.036413
25	3	46	5	Sí	4.257682	0.025611
0	1	126	5	No	4.860457	0.085483
0	2	126	5	No	2.530053	0.087958
0	3	126	4	No	2.365166	0.093950

Evidencias de las ejecuciones:

```
=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 3
Completadas: 3
No completadas: 0

Solicitud #1 (Texto (simulacion)): "Quiero agua"
Box: Meeseeks asociado (PID 3162, PPID 3156, N 1, i 1).
Box: ganador (PID 3162, PPID 3156, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Solicitud resuelta por el Meeseeks (PID 3162, PPID 3156, N 1, i 1).
Box: tiempo transcurrido de la solicitud: 0.505295 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.001270 segundos.

Solicitud #2 (Texto (simulacion)): "Quiero agua 2"
Box: Meeseeks asociado (PID 3164, PPID 3156, N 1, i 1).
Box: ganador (PID 3164, PPID 3156, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Solicitud resuelta por el Meeseeks (PID 3164, PPID 3156, N 1, i 1).
Box: tiempo transcurrido de la solicitud: 0.746402 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.001182 segundos.

Solicitud #3 (Texto (simulacion)): "Quiero agua 3"
Box: Meeseeks asociado (PID 3165, PPID 3156, N 1, i 1).
Box: ganador (PID 3165, PPID 3156, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Solicitud resuelta por el Meeseeks (PID 3165, PPID 3156, N 1, i 1).
Box: tiempo transcurrido de la solicitud: 0.504778 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.001013 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 46: Informe de las tres solicitudes con facilidad 95.

```
=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 3
Completadas: 3
No completadas: 0

Solicitud #1 (Texto (simulacion)): "Quiero una casa"
Box: Meeseeks asociado (PID 3180, PPID 3175, N 1, i 1).
Box: ganador (PID 3182, PPID 3180, N 2, i 2).
Box: solicitud completada.
Box: Meeseeks creados: 3. Profundidad maxima: 2.
Box: Solicitud resuelta por el Meeseeks (PID 3182, PPID 3180, N 2, i 2).
Box: tiempo transcurrido de la solicitud: 1.010167 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.002321 segundos.

Solicitud #2 (Texto (simulacion)): "Quiero una casa 2"
Box: Meeseeks asociado (PID 3183, PPID 3175, N 1, i 1).
Box: ganador (PID 3188, PPID 3187, N 4, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 11. Profundidad maxima: 5.
Box: Solicitud resuelta por el Meeseeks (PID 3188, PPID 3187, N 4, i 1).
Box: tiempo transcurrido de la solicitud: 2.048549 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.007845 segundos.

Solicitud #3 (Texto (simulacion)): "Quiero una casa 3"
Box: Meeseeks asociado (PID 3194, PPID 3175, N 1, i 1).
Box: ganador (PID 3194, PPID 3175, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Solicitud resuelta por el Meeseeks (PID 3194, PPID 3175, N 1, i 1).
Box: tiempo transcurrido de la solicitud: 0.512464 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.001000 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 47: Informe de las tres solicitudes con facilidad 65.

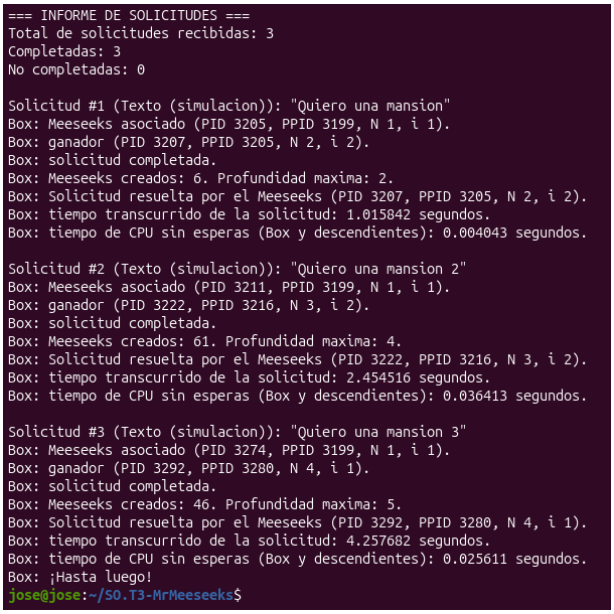


Figura 48: Informe de las tres solicitudes con facilidad 25.

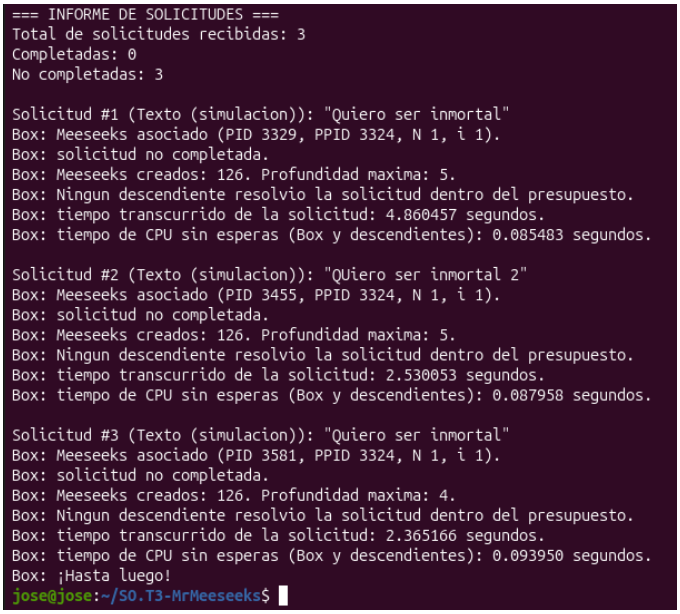


Figura 49: Informe de las tres solicitudes imposibles con facilidad 0.

Análisis de resultados:

Los resultados muestran una relación entre la facilidad de una solicitud y la cantidad de Meeseeks que se crean para intentar resolverla. Con una facilidad de 95, las tres corridas se completaron utilizando un solo proceso, con un tiempo promedio de 0.585492 segundos. En cambio, con facilidad 65, aunque todas las solicitudes se completaron, la cantidad de Meeseeks varió entre 1 y 11, alcanzando un promedio de 5 procesos por corrida. Esto demuestra que el resultado no siempre es igual, ya que depende también del componente aleatorio de la simulación.

La diferencia fue más notable con facilidad 25, donde se crearon 6, 61 y 46 Meeseeks respectivamente, alcanzando un promedio de 37.67 procesos y un tiempo promedio de 2.576013 segundos. Aunque las tres solicitudes se resolvieron, necesitaron más ayuda y alcanzaron hasta cinco niveles de profundidad. Por otra parte, las solicitudes con facilidad 0 nunca se completaron y generaron 126 Meeseeks en cada corrida, hasta que el presupuesto restante fue insuficiente para seguir creando grupos de cinco hijos.

En general, al disminuir la facilidad aumentaron la cantidad de procesos utilizados y el tiempo promedio de ejecución. También se observó que el tiempo de CPU fue mucho menor que el tiempo transcurrido, debido a las esperas simuladas y al trabajo concurrente de los procesos. Algunas diferencias de duración pueden estar relacionadas con la carga de la máquina virtual. Con estas pruebas se comprobó que el modelo probabilístico produce comportamientos distintos según la dificultad de la solicitud y que el límite de procesos permite controlar las tareas imposibles.

5.2. Experimento E2: comparación de operaciones y programas externos

Pregunta de investigación: ¿Qué diferencias existen entre procesar expresiones aritméticas y lógicas y ejecutar programas externos, considerando los tiempos de ejecución y el uso de CPU?

Para este experimento se utilizaron los resultados de las corridas B1, B2, B3, C1, C2 y C3, presentadas anteriormente. En todas las solicitudes se creó un único Meeseeks, con una profundidad máxima de 1. Se compararon los tiempos registrados y el estado final de cada solicitud.

Cuadro 4: Comparación entre operaciones y programas externos.

Caso	Solicitud	Estado	Tiempo (s)	CPU (s)
B1	`2 + 3 * 4`	Completada	0.000507	0.000552
B2	Expresión lógica	Completada	0.000552	0.000645
B3	`5 + * 2`	No completada	0.000598	0.000650
C1	`pwd`	Completada	0.449258	0.005700
C2	`ls /directorio6767`	No completada	0.011520	0.007869
C3	`sleep 10`	No completada	5.115131	0.011908

Análisis de resultados:

Al comparar los resultados se observó que las operaciones aritméticas y lógicas tuvieron tiempos de ejecución muy pequeños, cercanos a 0.0005 segundos. Las tres solicitudes utilizaron un solo Meeseeks y no necesitaron crear procesos adicionales. Además, cuando se ingresó una expresión incorrecta, el programa detectó el error sin afectar su funcionamiento.

En el caso de los programas externos, los tiempos fueron más variables. El comando `pwd` tardó 0.449258 segundos, mientras que `ls /directorio6767` terminó con un error en 0.011520 segundos. Por otra parte, `sleep 10` alcanzó el límite de cinco segundos y tuvo que ser cancelado. Aunque estos programas también utilizaron un solo Meeseeks, sus duraciones dependieron del comportamiento de cada ejecutable y de las condiciones del sistema.

También se observó que el tiempo de CPU fue mucho menor que el tiempo transcurrido en los programas externos, especialmente en `sleep 10`, porque el proceso permaneció esperando durante la mayor parte de su ejecución. En general, las pruebas muestran que ambos tipos de solicitudes funcionan correctamente, pero sus tiempos no son directamente comparables debido a que realizan trabajos diferentes. Además, algunas variaciones pueden estar relacionadas con el uso de una máquina virtual.

5.3. Experimento E3: contingencia por procesos y caos planetario

Pregunta de investigación: ¿Cómo se comporta el programa cuando una solicitud agota el presupuesto de Meeseeks o supera el tiempo máximo permitido?

Para este experimento se utilizaron los resultados de las corridas A4 y C3, además de las tres solicitudes imposibles realizadas en el experimento E1. Se compararon los dos mecanismos de contingencia implementados: el límite de procesos y la cancelación por tiempo. En todas las ejecuciones se mantuvo un presupuesto máximo de 128 Meeseeks por solicitud.

Cuadro 5: Comparación de los mecanismos de contingencia.

Escenario	Plazo	Creados	Prof.	Tiempo (s)	CPU (s)	Estado
A4: facilidad 0	300	126	5	3.150346	0.086134	No completada
E1: facilidad 0, corrida 1	300	126	5	4.860457	0.085483	No completada
E1: facilidad 0, corrida 2	300	126	5	2.530053	0.087958	No completada
E1: facilidad 0, corrida 3	300	126	4	2.365166	0.093950	No completada
C3: `sleep 10`	5	1	1	5.115131	0.011908	Caos planetario

Análisis de resultados:

En las solicitudes con facilidad 0 se observó que ningún Meeseeks logró resolver la tarea, ya que su probabilidad de éxito siempre fue del 0 %. En las cuatro ejecuciones se crearon 126 Meeseeks, alcanzando profundidades de cuatro y cinco niveles. Aunque el presupuesto permitido era de 128 procesos, los Meeseeks dejaron de crear hijos cuando ya no había suficientes recursos para formar los grupos de cinco que necesitaban. Los tiempos variaron entre 2.365166 y 4.860457 segundos, pero todas las solicitudes terminaron como no completadas.

Por otra parte, en la corrida C3 se ejecutó el comando `sleep 10` con un tiempo máximo de cinco segundos. Al superar este plazo, la Box declaró el caos planetario y canceló la ejecución. La solicitud terminó después de 5.115131 segundos y se registró como no completada. En este caso solamente se utilizó un Meeseeks, ya que no era necesario crear más procesos para ejecutar el programa externo.

Los resultados muestran que ambos mecanismos permiten controlar situaciones diferentes. El límite de procesos evita que una tarea imposible continúe creando Meeseeks sin control, mientras que el límite de tiempo permite detener una ejecución que tarda más de lo permitido. De esta manera, el programa puede finalizar las solicitudes que no logra resolver sin mantener procesos trabajando indefinidamente.

5.4. Experimento E4: ejecución después del plazo normal

Pregunta de investigación: ¿Qué sucede cuando se permite que una solicitud continúe trabajando después de superar el tiempo máximo normal?

Para este experimento se realizaron cuatro corridas. En la primera se utilizó el modo normal, mientras que en las tres restantes se activó el modo experimental. Este permite que los Meeseeks continúen trabajando después del plazo normal, siempre que no superen el límite extraordinario establecido.

En las primeras tres corridas se utilizó la solicitud “Quiero aprender F1”, con una facilidad de 25 y una duración de simulación de 1.5 segundos. En la última se ejecutó el programa externo `sleep 4`.

Cuadro 6: Resultados de las ejecuciones con y sin modo experimental.

Corrida	Modo	Normal	Extra	Meeseeks	Tiempo (s)	CPU (s)	Estado
E4.1	Normal	1	–	1	1.034351	0.001230	No completada
E4.2	Experimental	1	8	1	1.961870	0.001269	Completada
E4.3	Experimental	1	8	11	3.038427	0.005425	Completada
E4.4	Experimental	2	6	1	4.052283	0.010732	Completada

Evidencias de las ejecuciones:

```
Hi, I'm Mr. Meeseeks! Look at Meeeee. (4007, 3991, 1, 1)
Meeseeks (PID 4007, PPID 3991): Recibi por pipe la solicitud "Quiero aprender F1" (Texto (simulacion)).
Meeseeks (PID 4007, PPID 3991): facilidad efectiva 25, probabilidad 6.25%, simulacion 1.5000 segundos.
Meeseeks (PID 4007, PPID 3991): Caos planetario: se alcanzo el plazo de ejecucion.
Meeseeks (PID 4007, PPID 3991): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 4007, PPID 3991, N 1, i 1).
Box: solicitud no completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Caos planetario: se alcanzo el plazo de ejecucion.
Box: tiempo transcurrido de la solicitud: 1.034351 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.001230 segundos.

=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 1
Completadas: 0
No completadas: 1

Solicitud #1 (Texto (simulacion)): "Quiero aprender F1"
Box: Meeseeks asociado (PID 4007, PPID 3991, N 1, i 1).
Box: solicitud no completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Caos planetario: se alcanzo el plazo de ejecucion.
Box: tiempo transcurrido de la solicitud: 1.034351 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.001230 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 50: Informe final de la corrida E4.1.

```
Hi, I'm Mr. Meeseeks! Look at Meeeee. (4023, 4018, 1, 1)
Meeseeks (PID 4023, PPID 4018): Recibi por pipe la solicitud "Quiero aprender F1" (Texto (simulacion)).
Meeseeks (PID 4023, PPID 4018): facilidad efectiva 25, probabilidad 6.25%, simulacion 1.5000 segundos.
Box: plazo normal superado; modo experimental activo hasta 8.000 segundos.
Meeseeks (PID 4023, PPID 4018): Resolvi la solicitud; informare a mi padre por pipe.
Meeseeks (PID 4023, PPID 4018): Solicitud resuelta por el Meeseeks (PID 4023, PPID 4018, N 1, i 1).
Meeseeks (PID 4023, PPID 4018): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 4023, PPID 4018, N 1, i 1).
Box: ganador (PID 4023, PPID 4018, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Solicitud resuelta por el Meeseeks (PID 4023, PPID 4018, N 1, i 1).
Box: tiempo transcurrido de la solicitud: 1.961870 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.001269 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 0

=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 1
Completadas: 1
No completadas: 0

Solicitud #1 (Texto (simulacion)): "Quiero aprender F1"
Box: Meeseeks asociado (PID 4023, PPID 4018, N 1, i 1).
Box: ganador (PID 4023, PPID 4018, N 1, i 1).
Box: solicitud completada.
Box: Meeseeks creados: 1. Profundidad maxima: 1.
Box: Solicitud resuelta por el Meeseeks (PID 4023, PPID 4018, N 1, i 1).
Box: tiempo transcurrido de la solicitud: 1.961870 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.001269 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 51: Informe final de la corrida E4.2.

```
Meeseeks (PID 4040, PPID 4033): Recogí con waitpid al hijo PID 4044.
Meeseeks (PID 4040, PPID 4033): Solicitud resuelta por el Meeseeks (PID 4042, PPID 4040, N 2, i 2).
Meeseeks (PID 4040, PPID 4033): Gracias por permitirme ayudar. Hasta luego. Descendientes recogidos; resultado enviado al padre.
Box: Meeseeks asociado (PID 4040, PPID 4033, N 1, i 1).
Box: ganador (PID 4042, PPID 4040, N 2, i 2).
Box: solicitud completada.
Box: Meeseeks creados: 11. Profundidad maxima: 3.
Box: Solicitud resuelta por el Meeseeks (PID 4042, PPID 4040, N 2, i 2).
Box: tiempo transcurrido de la solicitud: 3.038427 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.005425 segundos.

--- MENU PRINCIPAL ---
1. Solicitud textual (simulacion)
2. Calculo aritmetico o logico
3. Ejecucion de programa externo
0. Salir
Opcion: 0

=== INFORME DE SOLICITUDES ===
Total de solicitudes recibidas: 1
Completadas: 1
No completadas: 0

Solicitud #1 (Texto (simulacion)): "Quiero aprender F1"
Box: Meeseeks asociado (PID 4040, PPID 4033, N 1, i 1).
Box: ganador (PID 4042, PPID 4040, N 2, i 2).
Box: solicitud completada.
Box: Meeseeks creados: 11. Profundidad maxima: 3.
Box: Solicitud resuelta por el Meeseeks (PID 4042, PPID 4040, N 2, i 2).
Box: tiempo transcurrido de la solicitud: 3.038427 segundos.
Box: tiempo de CPU sin esperas (Box y descendientes): 0.005425 segundos.
Box: ¡Hasta luego!
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 52: Informe final de la corrida E4.3.

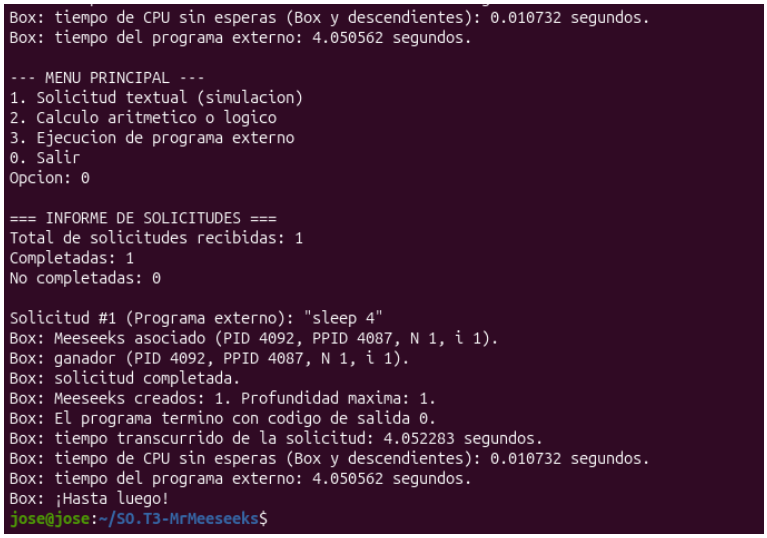


Figura 53: Informe final de la corrida E4.4.

Análisis de resultados:

En la primera corrida se estableció un plazo de un segundo para una solicitud cuya simulación duraba 1.5 segundos. Como el tiempo permitido era menor, el Meeseeks no logró terminar su intento y la Box declaró el caos planetario después de 1.034351 segundos. La solicitud quedó como no completada y solamente se creó un proceso.

En las dos corridas siguientes se activó el modo experimental, manteniendo el mismo plazo normal de un segundo, pero permitiendo continuar hasta ocho segundos. En la primera, el Meeseeks consiguió resolver la solicitud por sí mismo después de 1.961870 segundos, mientras que en la segunda necesitó ayuda de otros procesos y se crearon 11 Meeseeks, alcanzando tres niveles de profundidad. Esta última solicitud se completó después de 3.038427 segundos. La diferencia entre ambas ejecuciones se debe al comportamiento aleatorio de la simulación, ya que no siempre un Meeseeks consigue resolver la tarea en su primer intento.

Finalmente, se ejecutó el comando `sleep 4` con un plazo normal de dos segundos y un límite extraordinario de seis segundos. El programa continuó trabajando después del plazo normal y terminó correctamente en 4.052283 segundos, con un código de salida 0.

En general, los resultados muestran que el modo experimental permite continuar con solicitudes que normalmente serían interrumpidas por falta de tiempo. En estas pruebas, las tres ejecuciones experimentales lograron completarse, aunque necesitaron tiempos adicionales diferentes. Esto no garantiza que todas las tareas puedan resolverse, pero permite observar cuánto tiempo más necesitaron las solicitudes que sí tuvieron éxito. También se comprobó que el tiempo de CPU fue mucho menor que el tiempo transcurrido, debido principalmente a los periodos de espera de las simulaciones y del programa externo.

6. Uso de Git y organización de la entrega

El proyecto se mantiene en un repositorio de GitHub y los avances se registran mediante commits descriptivos. Esto permite identificar cambios de captura, procesos, comunicación, refactorización y pruebas, en lugar de presentar únicamente una copia final del código.

Repositorio de GitHub: `https://github.com/JoseMonge17/SO.T3-MrMeeseeks`

Commit identificado para la entrega: `c12844d`

```
jose@jose:~/SO.T3-MrMeeseeks$ git log --oneline --decorate
c12844d (HEAD -> main, origin/main, origin/HEAD) docs: Detalles código
6afe94d feat: Configuración Box
54ee559 fix: Tiempos de CPU
e8b807c chore: excluye ejecutables generados del repositorio
3fbb876 refactor: usa rand y srand para simplificar la simulacion
1760a64 docs: simplifica comentarios internos sin cambiar el codigo
ffc7e2c fix: Dos funciones iguales
6d3229b add: Documentación interna
fc846c7 feat: Interrupciones y experimentos
39b1c72 Implementación arbol recursivo y resolucion en cascada por pipes
aed3b50 feat: agregue simulacion individual, semaforos y limites por solicitud
d8d6268 feat: medir solicitudes y mostrar informe final
462648f feat: ejecutar programas externos y reportar su terminacion
b26c266 feat: evaluar calculos aritmeticos y logicos con TinyExpr
7bf34b0 feat: intercambiar solicitudes y resultados mediante pipes privados
26c15c8 feat: crear y recoger el primer proceso Meeseeks
7e06c0f fix: finalizar los menus ante EOF o error de lectura y reutilizar leer_numero
34d7b29 feat: Dificultad y tiempo de solicitudes textuales
ea9f06b feat: Agregar menu de seleccion de solicitudes
b5932c2 add: Recibir solicitudes
3b0391d chore: estructura inicial
jose@jose:~/SO.T3-MrMeeseeks$
```

Figura 54: Historial de commits del repositorio en GitHub.

7. Conclusiones

Con el desarrollo de esta tarea se logró implementar una aplicación en C que utiliza procesos de Linux para atender diferentes solicitudes. Mediante `fork()` se crearon los Meeseeks, mientras que los pipes permitieron comunicar las solicitudes, los resultados y los avisos de cancelación. También se utilizaron semáforos para proteger los contadores compartidos y `waitpid()` para recoger los procesos hijos. Las pruebas realizadas permitieron observar cómo estos mecanismos trabajan en conjunto para coordinar varios procesos.

En las solicitudes textuales se comprobó que la facilidad influye en la cantidad de Meeseeks necesarios para resolver una tarea. En el primer experimento, las solicitudes con facilidad 95 necesitaron solamente un proceso, mientras que las de facilidad 25 utilizaron un promedio de 37.67 Meeseeks. También se observó que los resultados pueden variar entre ejecuciones debido al comportamiento aleatorio de la simulación. Por otra parte, las tareas con facilidad 0 nunca se resolvieron y terminaron al agotarse el presupuesto disponible.

Las pruebas de operaciones aritméticas y lógicas mostraron que el programa puede interpretar expresiones y detectar errores sin detener el funcionamiento de la Box. De igual forma, se comprobó que los Meeseeks pueden ejecutar programas externos y revisar sus códigos de salida. Al comparar los tiempos, se observó que las operaciones matemáticas se resolvieron rápidamente, mientras que los programas externos presentaron duraciones diferentes según su comportamiento. Además, el tiempo de CPU fue generalmente menor que el tiempo transcurrido, especialmente cuando existían periodos de espera.

Otro resultado importante fue comprobar la utilidad de los límites de procesos y de tiempo. En las solicitudes imposibles, el presupuesto evitó que los Meeseeks continuaran creando descendientes indefinidamente. Asimismo, en la ejecución de `sleep 10` se observó cómo la Box declaró el caos planetario al superar los cinco segundos permitidos. Estos mecanismos permiten controlar los recursos y finalizar solicitudes que no pueden completarse dentro de las condiciones establecidas.

Finalmente, el modo experimental permitió observar qué sucede cuando una solicitud continúa después del plazo normal. En las dos pruebas textuales realizadas con este modo, la tarea logró completarse después de 1.961870 y 3.038427 segundos, respectivamente. También se consiguió ejecutar `sleep 4` con un plazo normal de dos segundos, terminando correctamente en 4.052283 segundos. Estas pruebas mostraron que disponer de más tiempo puede permitir completar algunas solicitudes, aunque el resultado también depende de la probabilidad de éxito y de los recursos disponibles.

8. Recomendaciones y trabajo futuro

Como recomendación, sería conveniente realizar una mayor cantidad de pruebas para cada nivel de facilidad, ya que el comportamiento probabilístico puede producir resultados diferentes incluso cuando se utilizan los mismos valores. Esto permitiría obtener promedios más representativos y comparar con mayor precisión la cantidad de procesos y los tiempos necesarios para resolver las solicitudes.

También sería útil repetir algunos experimentos directamente en un equipo con Linux, sin utilizar una máquina virtual, para comparar los tiempos obtenidos. Durante las pruebas se observaron algunas variaciones en la duración de las ejecuciones que podrían estar relacionadas con la carga del sistema y los recursos disponibles en la máquina virtual.

Como posible mejora futura, se podría organizar la salida de los Meeseeks de manera que los mensajes de cada proceso sean más fáciles de seguir cuando se crean varias generaciones al mismo tiempo. Esto facilitaría la observación del árbol de procesos y de la comunicación en cascada, especialmente en las solicitudes difíciles que generan muchos descendientes.

A. Código fuente

El código fuente completo se adjunta por separado junto con este informe. El archivo principal es `src/main.c`, desarrollado en lenguaje C para Linux. La biblioteca TinyExpr, utilizada en el caso de las expresiones, se encuentra en `vendor/tinyexpr`.

Los archivos fuente, las dependencias necesarias y el historial de cambios también están disponibles en el repositorio de GitHub

En las secciones de implementación se incluyeron capturas de las funciones principales para explicar la creación de procesos, la comunicación por pipes, la sincronización y el control de recursos.

Referencias

[1] N. Matthew y R. Stones, *Beginning Linux Programming*, 4.ª ed. Indianapolis, IN, EE. UU.: Wiley Publishing, Inc., 2007, caps. 13 y 14.

[2] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 9.ª ed. Hoboken, NJ, EE. UU.: John Wiley & Sons, 2013, cap. 3.

[3] B. W. Kernighan y D. M. Ritchie, *The C Programming Language*, 2.ª ed. Englewood Cliffs, NJ, EE. UU.: Prentice Hall, 1988.

[4] Codeplea, "TinyExpr: A tiny recursive descent expression parser and evaluation engine for C," GitHub. [En línea]. Disponible en: `https://github.com/codeplea/tinyexpr`. [Consultado: 14-oct-2026].